

[Get Started](#)

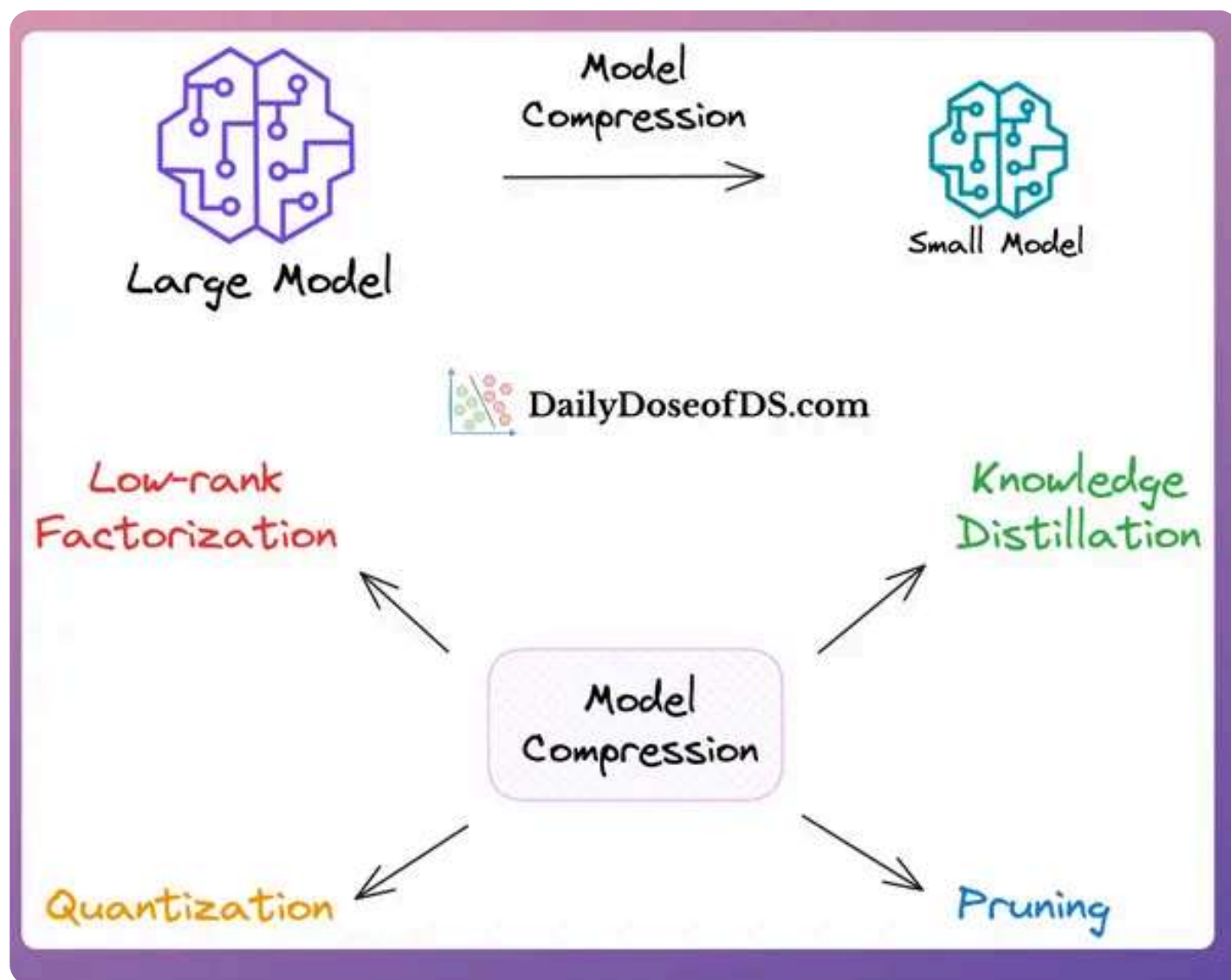
Sep 17, 2023

# Model Compression: A Critical Step Towards Efficient Machine Learning

Four critical ways to reduce model footprint and inference time.



Avi Chawla

[Join Today!](#)



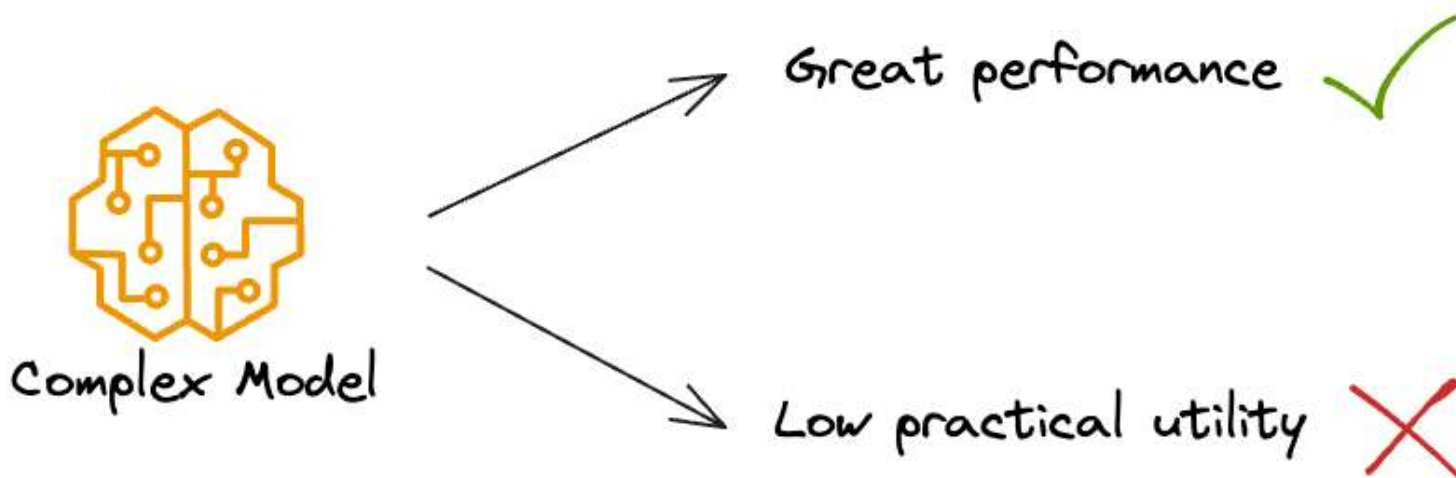
Hey! Enjoy the free article. It looks like you are from **India IN**. So whenever you are ready, join using this [membership link](#) for relief pricing of **50%** off on your subscription, FOREVER.

## Motivation

Training machine learning (ML) models are frequently driven by a relentless pursuit of achieving higher and higher accuracies.

Many create increasingly complex deep learning models, which, without a doubt, do incredibly well “performance-wise.”

However, the complexity severely impacts their real-world utility.

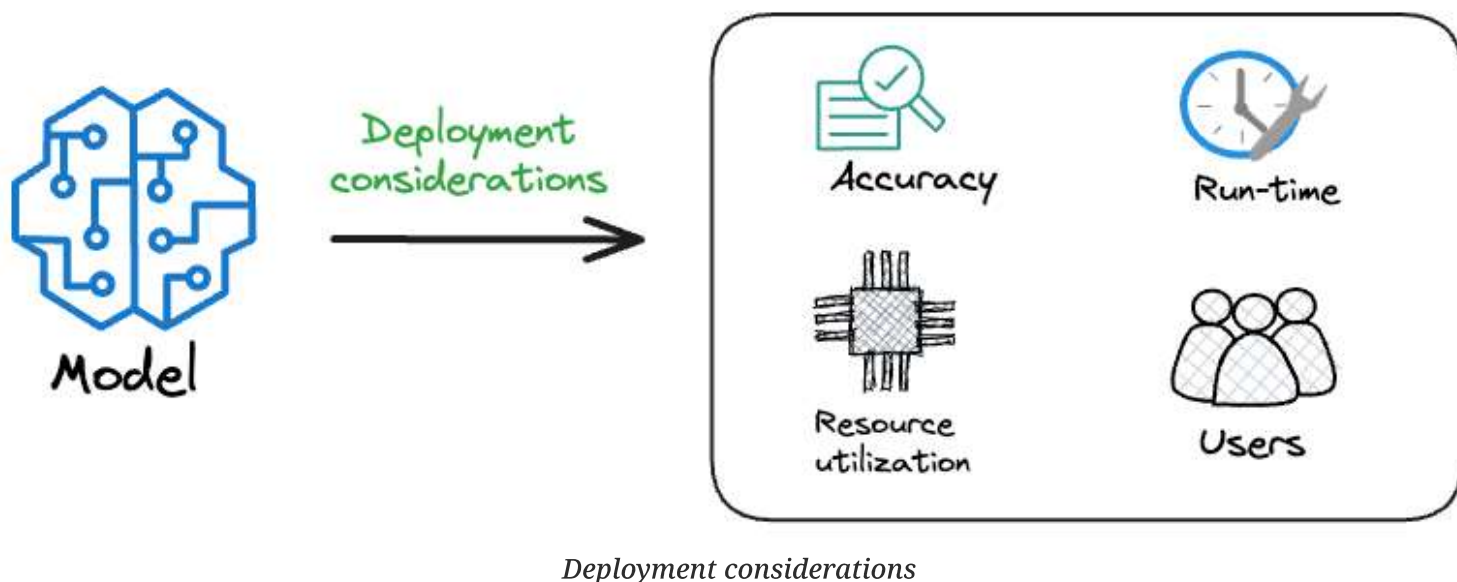


For years, the primary objective in model development has been to achieve the best performance metrics.

👉 This, unfortunately, is also a practice that many leaderboard-based competitions promote. Nothing wrong, but in my opinion, this overshadows the importance of focusing on the real-world applicability of the solution.

However, it is important to note that when it comes to deploying these models in production (or user-facing) systems, the focus shifts from raw accuracy to considerations such as efficiency, speed, and resource consumption.

Thus, typically, when we deploy any model to production, the specific model that gets shipped to production is NOT solely determined based on performance.



Instead, we must consider several operational metrics that are not ML-related.

**What are they? Let's understand!**

## Typical operational metrics

When a model is deployed into production, certain requirements must be met.

Typically, these “requirements” are not considered during the prototyping phase of the model.

For instance, it is fair to assume that a user-facing model may have to handle plenty of requests from a product/service the model is integrated with.

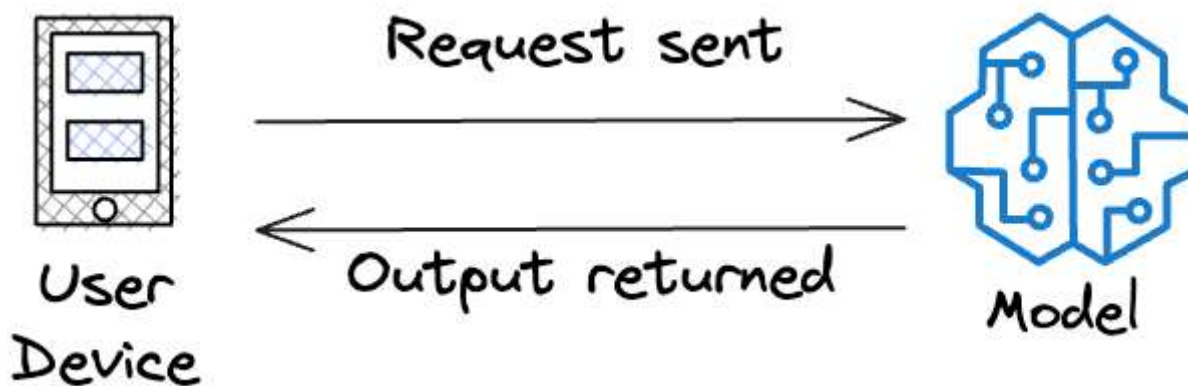
And, of course, we can never ask users to wait for, say, a minute for the model to run and generate predictions.

Thus, along with “model performance,” we would want to optimize for several other operational metrics:

## #1) Inference latency

It's the time it takes for a model to process a single input and generate a prediction.

It measures the delay between sending a request to the model and receiving the response.



Striving for a low inference latency is crucial for all real-time or interactive applications, as users expect a quick response.

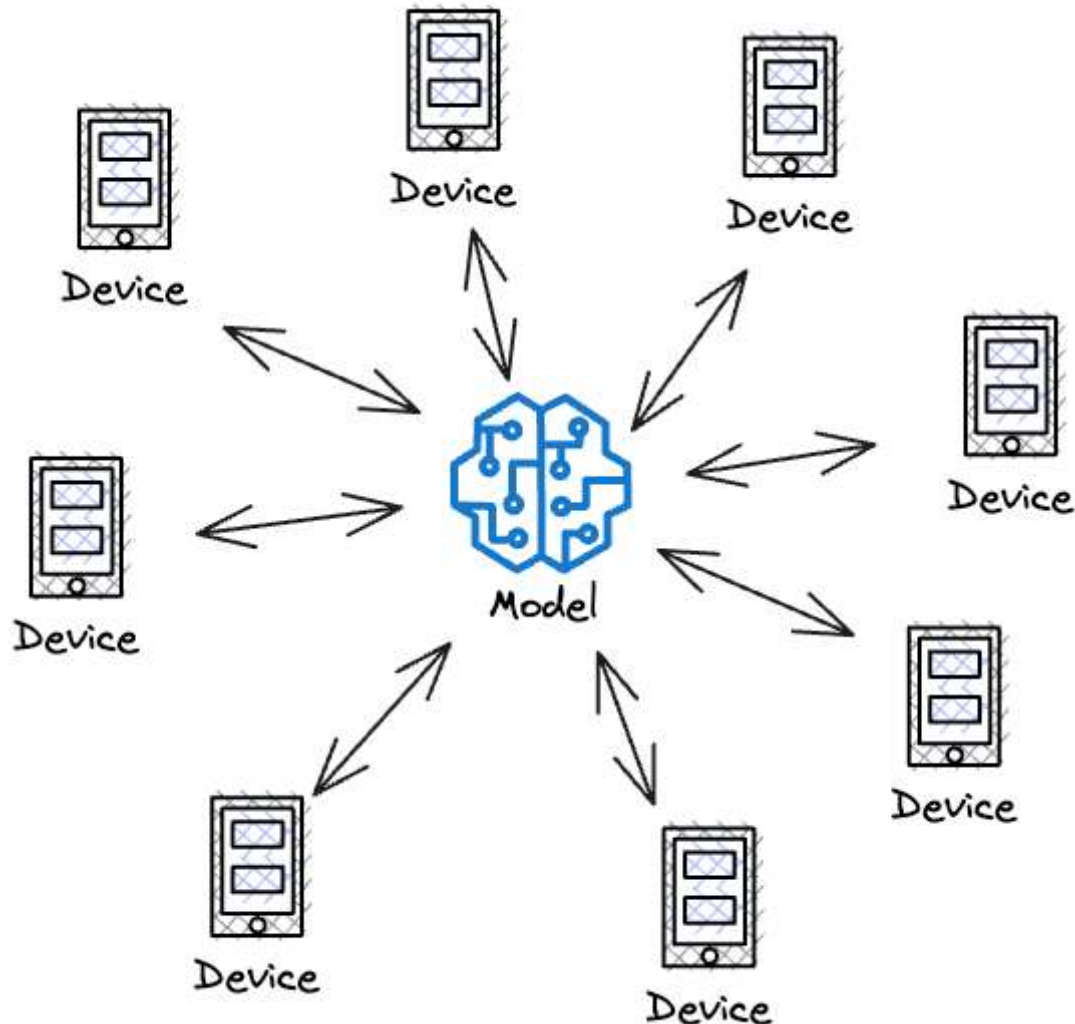
High latency, as you may have guessed, will lead to a poor user experience and will not be suitable for many applications like:

- Chatbots
- Real-time speech-to-text transcription
- Gaming, and many more.

## #2) Throughput

Throughput is the number of inference requests a model can handle in a given time period.

It estimates the model's ability to process multiple requests simultaneously.



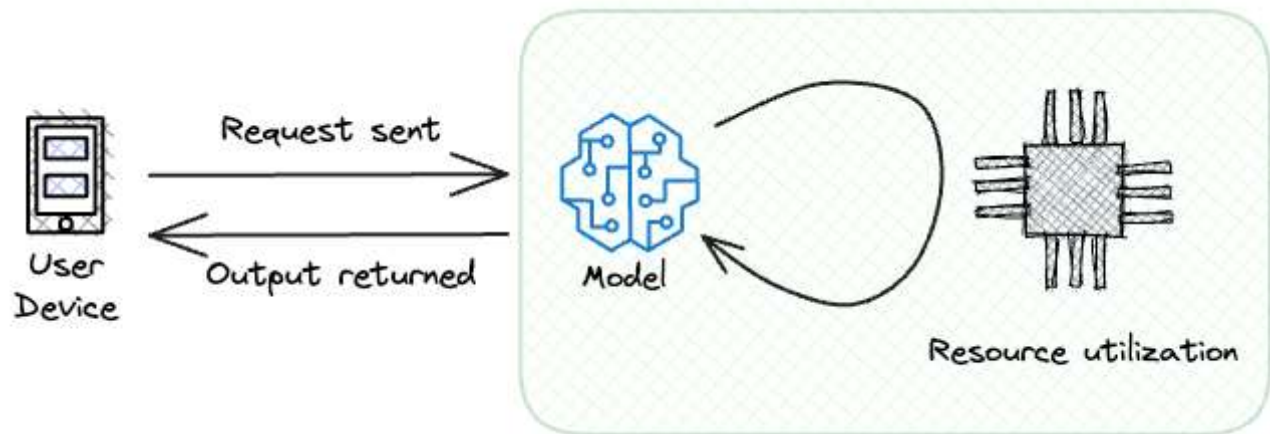
Yet again, as you may have guessed, high throughput is essential for applications with a high volume of incoming requests.

These include e-commerce websites, recommendation systems, social media platforms, etc. High throughput ensures that the model can serve many users concurrently without significant delays.

### #3) Model size

This refers to the amount of memory a model occupies when loaded for inference purposes.

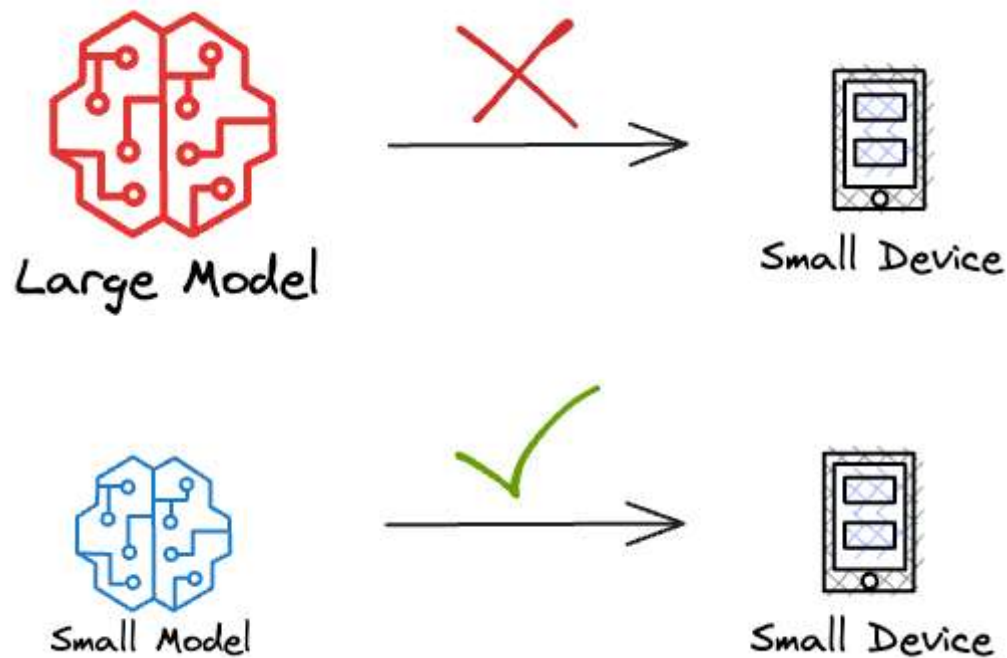
It quantifies the memory footprint required to store all the parameters, configurations, and related data necessary for the model to make predictions or generate real-time outputs.



The significance of model size becomes particularly apparent when deploying models in resource-constrained environments.

Many production environments, such as mobile devices, edge devices, or IoT devices, have limited memory capacity.

It is obvious to guess that in such cases, the model's size will directly impact whether it can be deployed at all.



Large models may not fit within the available memory, making them impractical for these resource-constrained settings.

## A real-life instance

This is a famous story.

In 2006, Netflix launched the “Netflix Prize,” a machine learning competition that encouraged ML engineers to build the best algorithm to predict user ratings for films.

**The grand prize was USD 1, 000, 000.**

After the competition concluded, Netflix awarded a \$1 million prize to a developer team in 2009 for an algorithm that increased the accuracy of the company's recommendation engine by **10 percent**.

That’s a lot!

Yet, Netflix never used that solution because it was overly complex.

Here's what Netflix said:

---

*The increase in accuracy of the winning improvements did not seem to justify the engineering effort needed to bring them into a production environment.*

---

The complexity and resource demands of the developed model made it impractical for real-world deployment. Netflix faced several challenges:

1. **Scalability:** The model was not easily scalable to handle the vast number of users and movies on the Netflix platform. It would have required significant computational resources to make real-time recommendations for millions of users they had.
2. **Maintenance:** Managing and updating such a complex model in a production environment would have been a logistical nightmare. Frequent updates and changes to the model would be challenging to implement and maintain.
3. **Latency:** The ensemble model's inference latency was far from ideal for a streaming service. Users expect near-instantaneous recommendations, but the complexity of the model made achieving low latency difficult.

*You can read more about this story here: [Netflix Prize story](#).*

Consequently, Netflix never integrated the winning solution into its production recommendation system. Instead, they continued to use a simplified version of their existing algorithm, which was more practical for real-time recommendations.

This real-life instance from the Netflix Prize was a reminder that we must strive for a delicate balance between model complexity and practical utility.

**While highly complex models may excel in research and competition settings, they may not be suitable for real-world deployment due to scalability, maintenance, and latency concerns.**

In practice, simpler and more efficient models often are a better choice for delivering a seamless user experience in production environments.

Let me ask you this. Which of the following two models would you prefer to integrate into a user-facing product?



Model A

Accuracy: 99%

Run-time: 2 seconds

Size: 125 MBs



Model B

Accuracy: 97%

Run-time: 0.1 seconds

Size: 10 MBs

I strongly prefer Model B.

If you understand this, you resonate with the idea of keeping things simple in production.

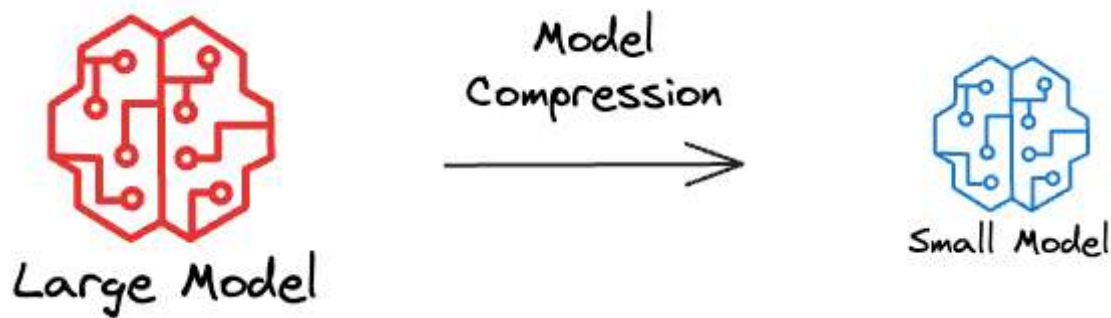
Fortunately, there are various techniques that can help us reduce the size of the model, thereby increasing the speed of model inference.

These techniques are called **Model Compression** methods.

Using these techniques, you can reduce both the latency and size of the original model.

## Model Compression

As the name suggests, model compression is a set of techniques used to reduce the size and computational complexity of a model while preserving or even improving its performance.

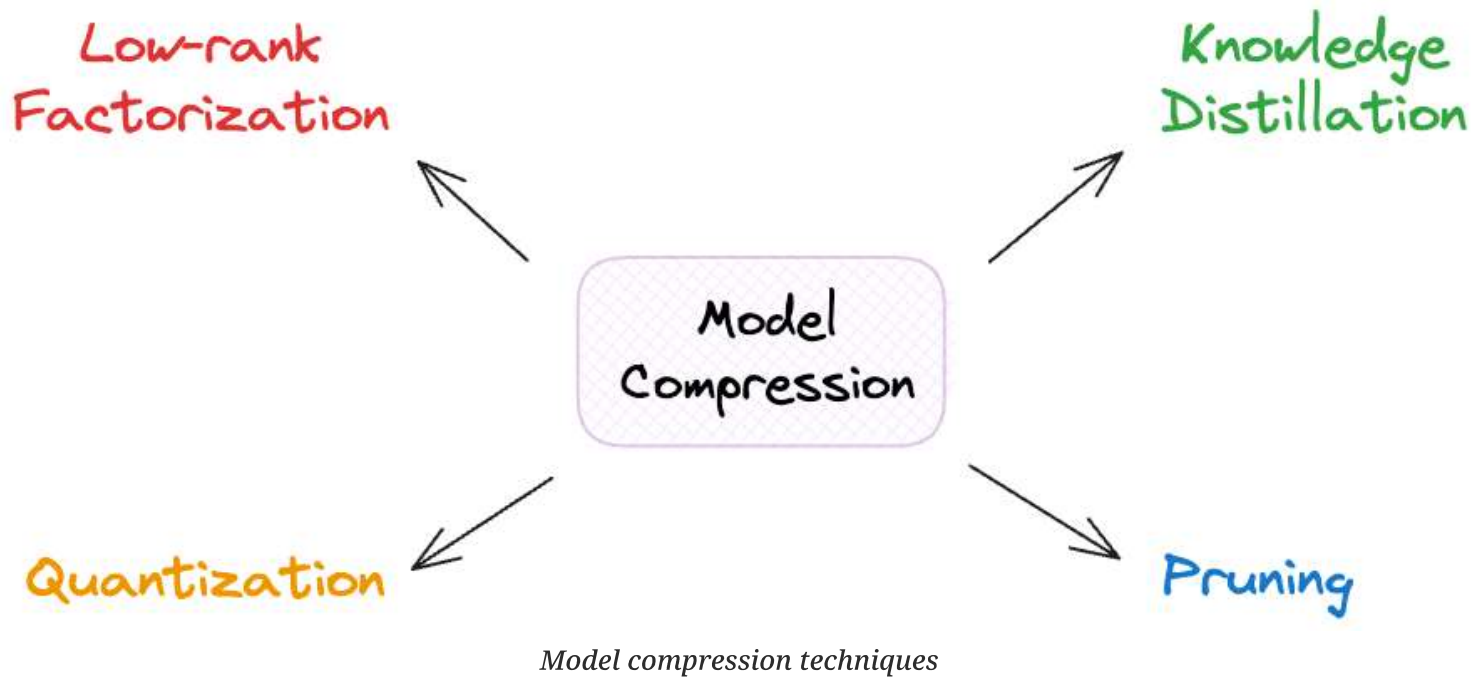


They aim to make the model smaller — that is why the name “**model compression**.”

Typically, it is expected that a smaller model will:

- Have a lower inference latency as smaller models can deliver quicker predictions, making them well-suited for real-time or low-latency applications.
- Be easy to scale due to their reduced computational demands.
- Have a smaller memory footprint.

In this article, we’ll look at four techniques that help us achieve this:



1. Knowledge Distillation
2. Pruning
3. Low-rank Factorization
4. Quantization

As we will see shortly, these techniques attempt to strike a balance between model size and accuracy, making it relatively easier to deploy models in user-facing products.

👉 The Jupyter notebook of this entire article has been provided at the bottom of the article.

Let's understand them one by one!

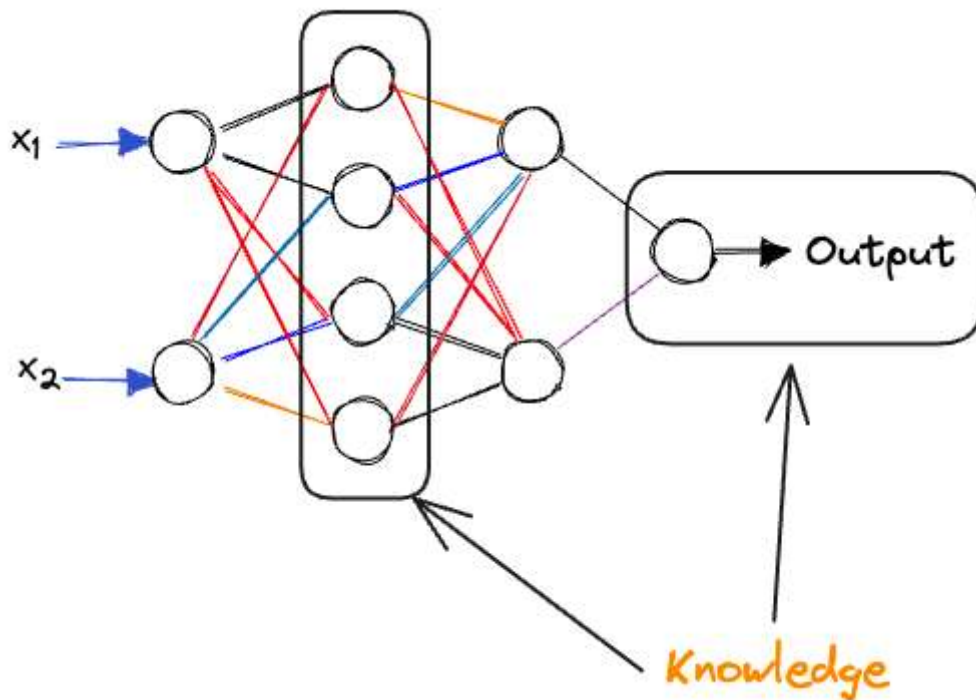
## #1) Knowledge Distillation

This is one of the most common, effective, reliable, and one of my favorite techniques to reduce model size.

Essentially, knowledge distillation involves training a smaller, simpler model (referred to as the “student” model) to mimic the behavior of a larger, more complex model (known as the “teacher” model).

The term can be broken down as follows:

- **Knowledge:** Refers to the understanding, insights, or information that a machine learning model has acquired during training. This “knowledge” can be typically represented by the model’s parameters, learned patterns, and its ability to make predictions.



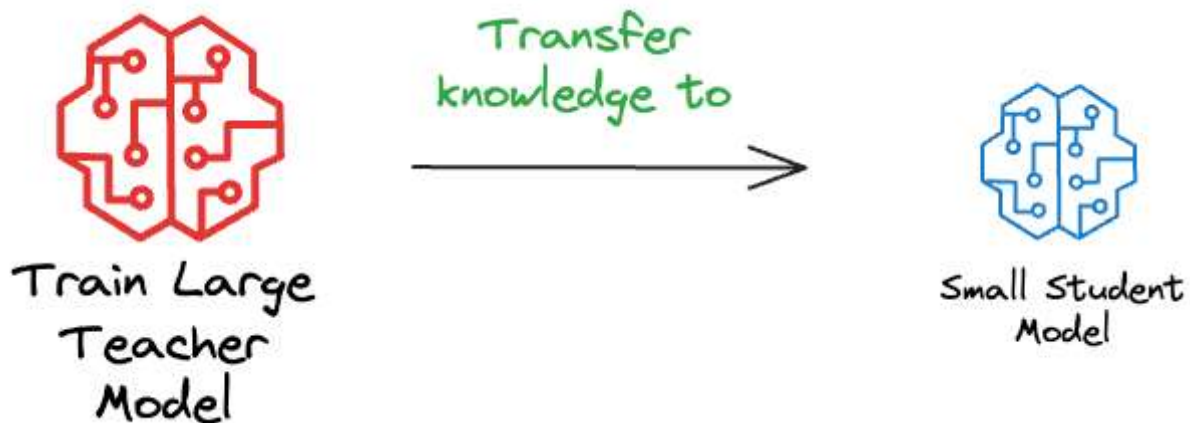
*Knowledge of a neural network*

- **Distillation:** In this context, distillation means transferring or condensing knowledge from one model to another. It involves training the student model to mimic the behavior of the teacher model, effectively transferring the teacher's knowledge.



This is a two-step process:

- Train the large model as you typically would. This is called the “teacher” model.
- Train a smaller model, which is intended to mimic the behavior of the larger model. This is also called the “student” model.



The primary objective of knowledge distillation is to transfer the knowledge, or the learned insights, from the teacher to the student model.

This allows the student model to achieve comparable performance with fewer parameters and reduced computational complexity.

The technique makes intuitive sense as well.

Of course, comparing it to a real-world teacher-student scenario in an academic setting, the student model may never perform as well as the teacher model.

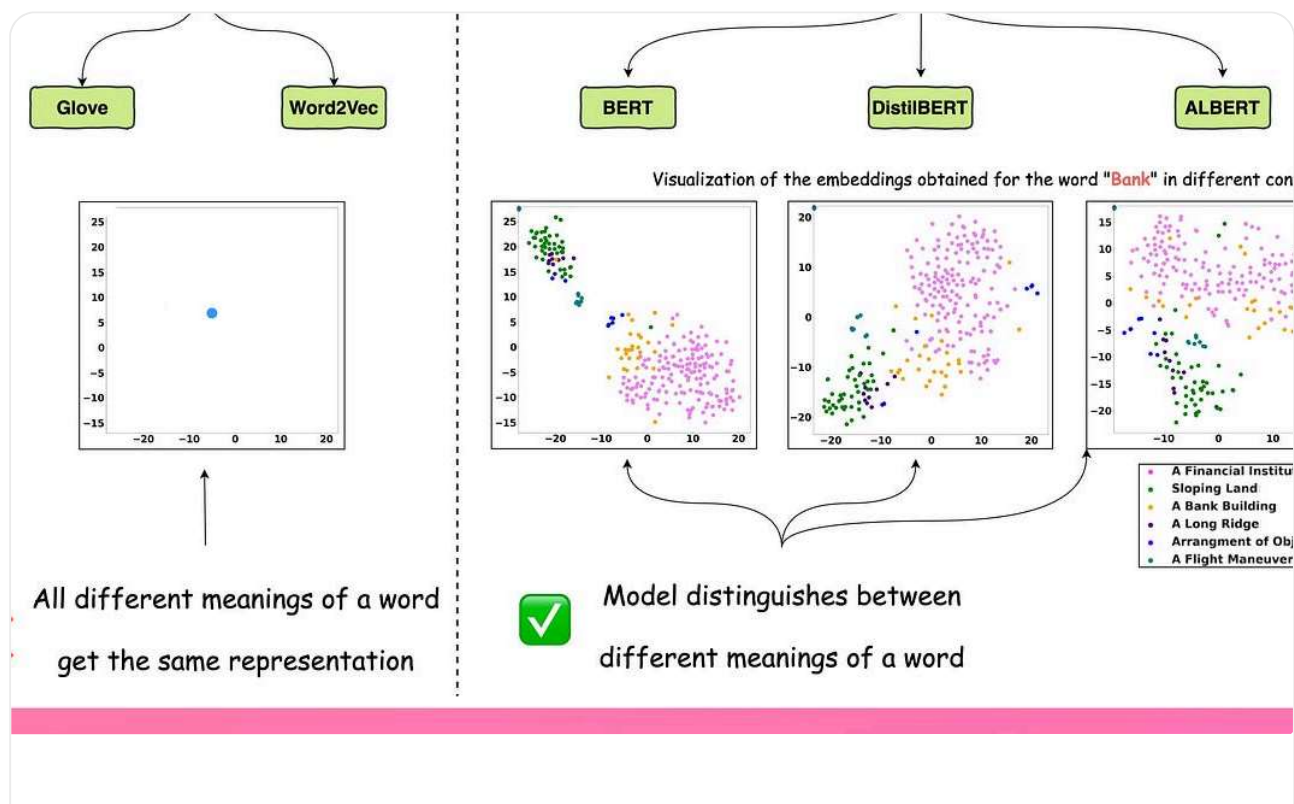
But with consistent training, we can create a smaller model that is **almost** as good as the larger one.

This goes back to the objective we discussed above

*Strike a balance between model size and accuracy, such that it is relatively easier to deploy models in user-facing products.*

A classic example of a model developed in this way is [DistillBERT](#). It is a student model of [BERT](#).

We also discussed this in the newsletter here:



## The Limitation of Static Embeddings Which Made Them Obsolete

A visual guide to context-aware embeddings.

 Daily Dose of Data Science • Avi Chawla

DistilBERT is approximately 40% smaller than BERT, which is a massive difference in size.

Still, it retains approximately 97% of the natural language understanding (NLU) capabilities of BERT.

**What's more, DistilBERT is roughly 60% faster in inference.**

This is something I have personally experienced and verified in one of my research studies on Transformer models:

| Model      | SensEval-2 |       |       |       |       |       | SensEval-3 |       |       |       |       |       |
|------------|------------|-------|-------|-------|-------|-------|------------|-------|-------|-------|-------|-------|
|            | k=1        | k=3   | k=5   | k=7   | k=10  | k=11  | k=1        | k=3   | k=5   | k=7   | k=10  | k=11  |
| BERT       | 76.02      | 76.78 | 76.62 | 76.62 | 76.76 | 76.81 | 79.40      | 80.31 | 80.49 | 80.96 | 80.75 | 80.72 |
| DistilBERT | 74.81      | 75.64 | 75.36 | 75.43 | 75.41 | 75.43 | 78.62      | 79.71 | 80.05 | 80.15 | 80.23 | 80.07 |

As shown above, on one of the studied datasets (SensEval-2), BERT achieved the best accuracy of 76.81. With DistilBERT, it was 75.64.

On another task (SensEval-3), BERT achieved the best accuracy of 80.96. With DistilBERT, it was 80.23.

Of course, DistilBERT isn't as good as BERT. Yet, the performance difference is small.

Given the run-time performance benefits, it makes more sense to proceed with DistilBERT instead of BERT in a production environment.



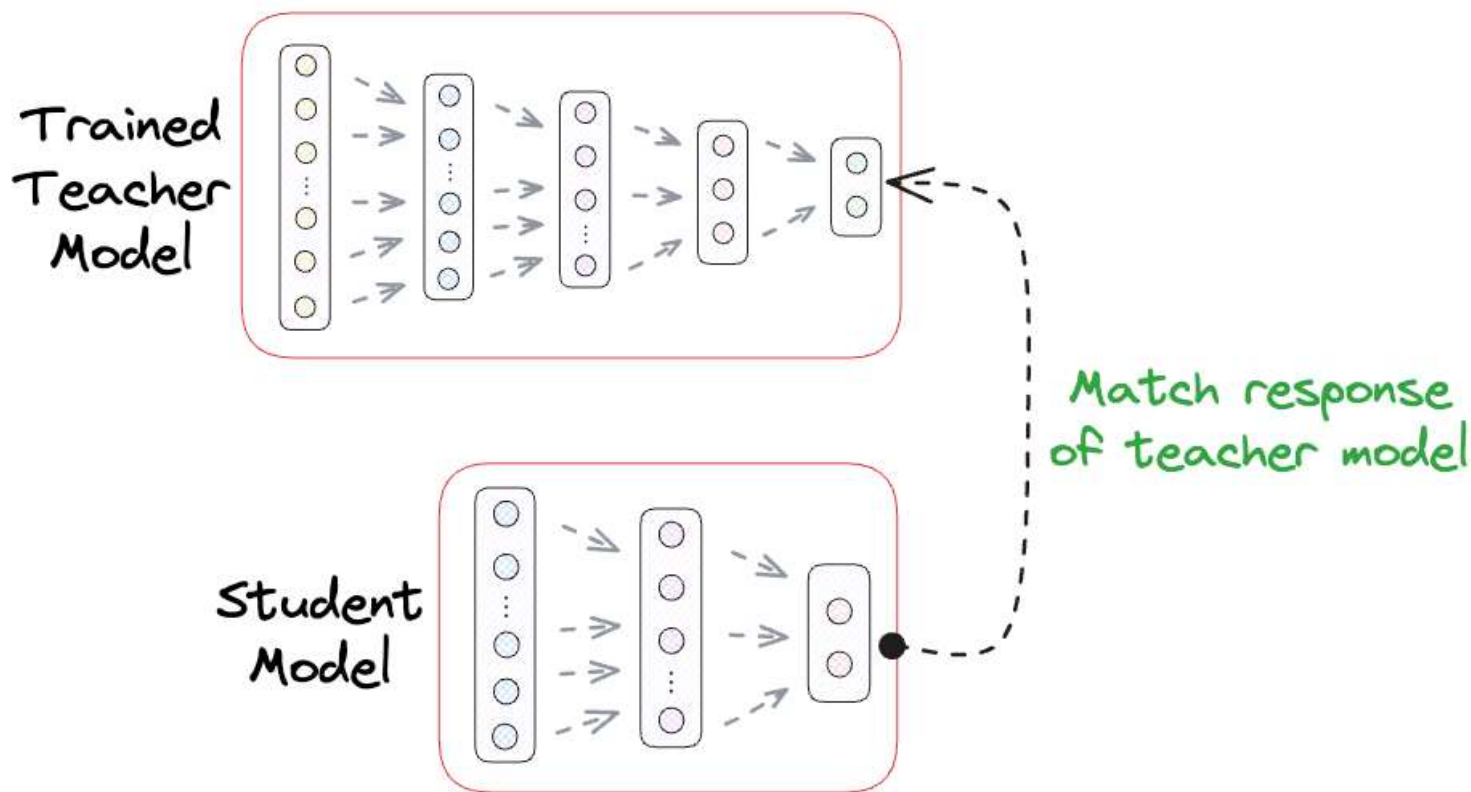
If you are interested in learning more about my research study, you can read it here: [A Comparative Study of Transformers on Word Sense Disambiguation](#).

One of the biggest downsides of knowledge distillation is that one must still train a larger teacher model first to train the student model.

However, in a resource-constrained environment, it may not be feasible to train a large teacher model.

Assuming we are not resource-constrained at least in the development environment, one of the most common techniques for Knowledge Distillation is **Response-based Knowledge Distillation**.

As the name suggests, in **response-based knowledge distillation**, the focus is on matching the output responses (predictions) of the teacher model and the student model.



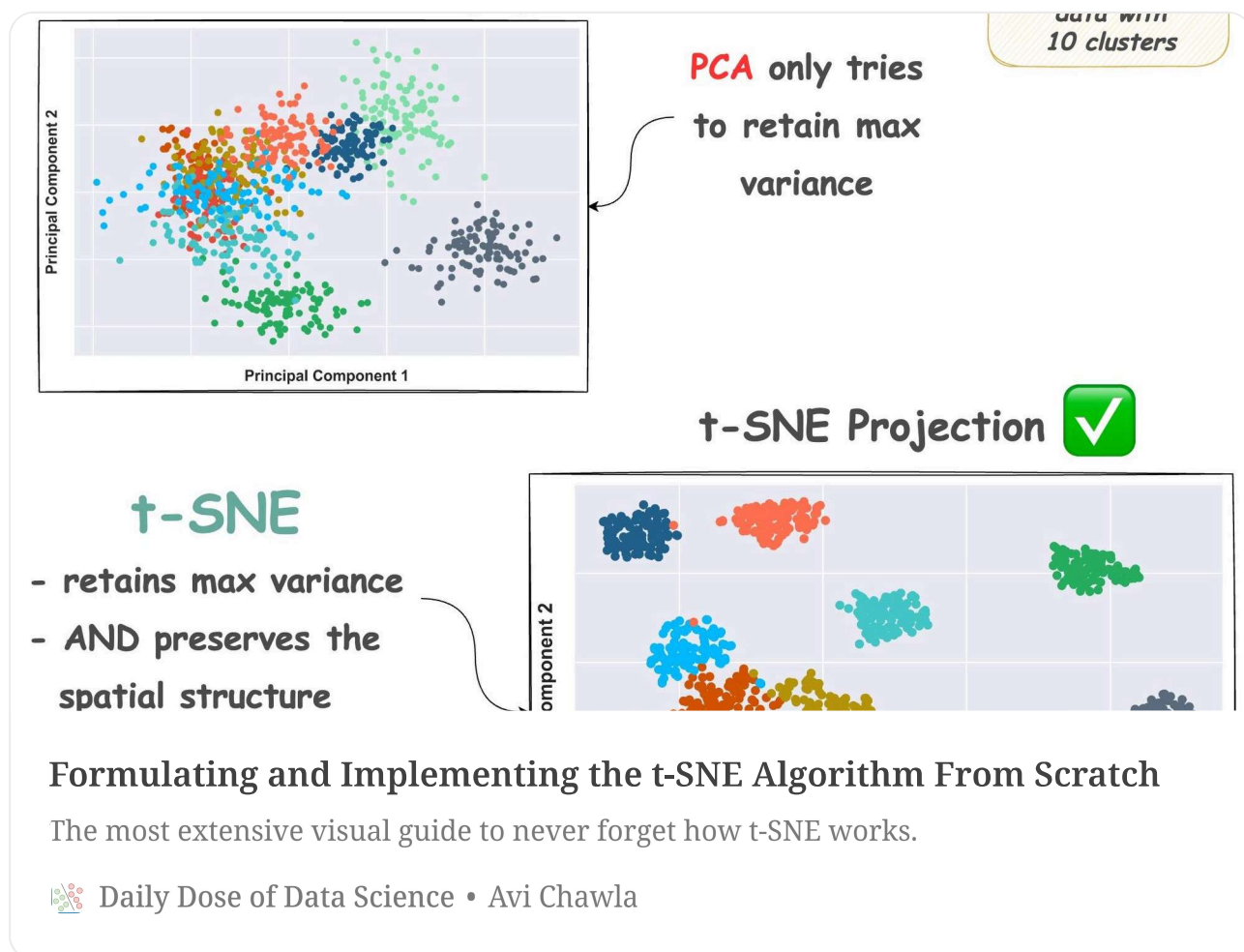
*Response-based Knowledge Distillation*

Talking about a classification use case, this technique transfers the **probability distributions** of class predictions from the teacher to the student.

It involves training the student to produce predictions that are not only accurate but also mimic the soft predictions (probability scores) of the teacher model.

As we are trying to mimic the **probability distribution of the class predictions of the teacher model**, one ideal candidate for the loss function is KL divergence.

We discussed this in detail in one of the previous articles on t-SNE.



Yet, here's a quick recap:

## KL divergence

The core idea behind KL divergence is to assess how much information is lost when one distribution is used to approximate another.

Thus, the more information is lost, the more the KL divergence. As a result, the more the dissimilarity.

KL divergence between two probability distributions  $P(x)$  and  $Q(x)$  is calculated as follows:

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \cdot \log \left( \frac{P(x)}{Q(x)} \right)$$

The formula for KL divergence can be read as follows:

The KL divergence  $D_{KL}(P||Q)$  between two probability distributions  $P$  and  $Q$  is calculated by summing the above quantity over all possible outcomes  $x$ . Here:

- $P(x)$  represents the probability of outcome  $x$  occurring according to distribution  $P$ .
- $Q(x)$  represents the probability of the same outcome occurring according to distribution  $Q$ .

It measures how much information is lost when using distribution  $Q$  to approximate distribution  $P$ .

Imagine this. Say  $P$  and  $Q$  were identical. This should result in zero loss of information. Let's verify this from the formula above.

If the probability distributions  $P$  and  $Q$  are identical, it means that for every  $x$ ,  $P(x) = Q(x)$ . Thus,

$$\frac{P(x)}{Q(x)} = 1$$

or  $\log\left(\frac{P(x)}{Q(x)}\right) = 0$

Simplifying, we get:

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \cdot \log\left(\frac{P(x)}{Q(x)}\right)$$

$$= \sum_x P(x) \cdot 0$$

$$= 0$$

no loss of information

This is precisely what we intend to achieve in response-based knowledge distillation.

Simply put, we want the probability distribution of the class predictions of the student model to be identical to the probability distribution of the class predictions of the teacher model.

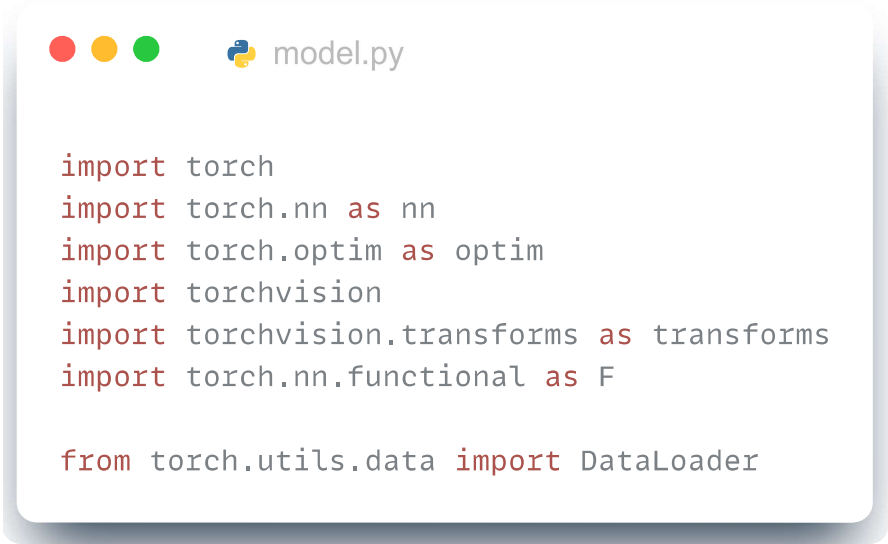
- First, we can train the teacher model as we typically would.
- Next, we can instruct the student model to mimic the probability distribution of the class predictions of the teacher model.

Let's see how we can practically use response-based knowledge distillation using PyTorch.

More specifically, we shall train a slightly complex neural network on the MNIST dataset. Then, we will build a simpler neural network using the response-based knowledge distillation technique.

## Step 1) Import required packages and libraries

First, we import the required packages from PyTorch:



```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torch.nn.functional as F

from torch.utils.data import DataLoader
```

## Step 2) Load the dataset

Next, we load the MNIST dataset (train and test) and create their respective PyTorch dataloaders.

```
model.py

# 1) Data Transformer
transform = transforms.Compose([transforms.ToTensor(),
                                transforms.Normalize((0.5,), (0.5,))])

# 2) Create Train Dataset
trainset = torchvision.datasets.MNIST(root='./data', train=True,
                                       download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=64, shuffle=True)

# 3) Create Test Dataset
testset = torchvision.datasets.MNIST(root='./data', train=False,
                                       download=True, transform=transform)
testloader = DataLoader(testset, batch_size=64, shuffle=False)
```

## Step 3) Define the Teacher Model

Now, we shall define a simple CNN-based neural network architecture. This is demonstrated below:

```
model.py

class TeacherNet(nn.Module):
    def __init__(self):
        super(TeacherNet, self).__init__()
        self.conv = nn.Conv2d(1, 32, 5)
        self.pool = nn.MaxPool2d(5, 5)
        self.fc1 = nn.Linear(32 * 4 * 4, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv(x))
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

## Step 4) Initialize the model and define the loss function and optimizer

Moving on, we shall initialize the teacher model and define the loss function to train it — the `CrossEntropyLoss`.



```
# Initialize model
teacher_model = TeacherNet()

# Define optimizer
teacher_optimizer = optim.Adam(teacher_model.parameters(),
                                lr=0.001)

# Define loss function
teacher_criterion = nn.CrossEntropyLoss()
```

## Step 5) Train the Teacher model

Now, we will train the teacher model.

 TeacherModel.py

```
for epoch in range(5):
    epoch_loss = 0.0

    # set to train mode
    teacher_model.train()

    # train for all batches of data
    for data in trainloader:
        ...

    # set to evaluation mode
    teacher_model.eval()
    teacher_accuracy = evaluate(teacher_model)

    # print performance metrics
    print(f"""Epoch {epoch + 1},
            Loss: {epoch_loss / len(testloader)},
            Acc: {teacher_accuracy * 100:.2f}
            """)
```



|          |             |             |
|----------|-------------|-------------|
| Epoch 1, | Loss: 0.20, | Acc: 98.79% |
| Epoch 2, | Loss: 0.17, | Acc: 98.91% |
| Epoch 3, | Loss: 0.15, | Acc: 98.78% |
| Epoch 4, | Loss: 0.13, | Acc: 98.89% |
| Epoch 5, | Loss: 0.11, | Acc: 98.63% |

With this, we are done with the Teacher model.

Next, we must train the Student Model.

## Step 6) Define the Student Model

We defined the Teacher model as a CNN-based neural network architecture. Let's define the Student model as a simple feed-forward neural network without any CNN layers:

```
StudentModel.py

class StudentNet(nn.Module):
    def __init__(self):
        super(StudentNet, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

No Convolution  
layer

## Step 7) Define the KL Divergence loss function

```
# Knowledge distillation loss (KL divergence)
def KL_loss(student_logits, teacher_logits):

    # convert teacher model outputs to probabilities
    p_teacher = F.softmax(teacher_logits, dim=1)

    # convert student model outputs to probabilities
    p_student = F.log_softmax(student_logits, dim=1)

    # compute KL divergence loss (PyTorch's method)
    loss = F.kl_div(p_student, p_teacher, reduction='batchmean')

    return loss
```

The above method accepts two parameters:

- The output of the student model ( `student_logits` ).

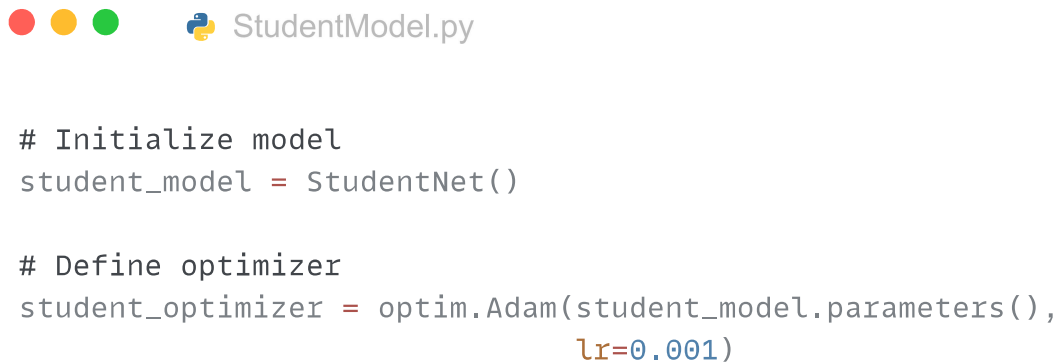
- The output of the teacher model ( `teacher_logits` ).

We convert both outputs to probabilities using the softmax function.

Finally, we find the KL divergence between them and return it.

## Step 8) Initialize the model and define the optimizer

Moving on, we shall initialize the student model and define the optimizer as we did before.



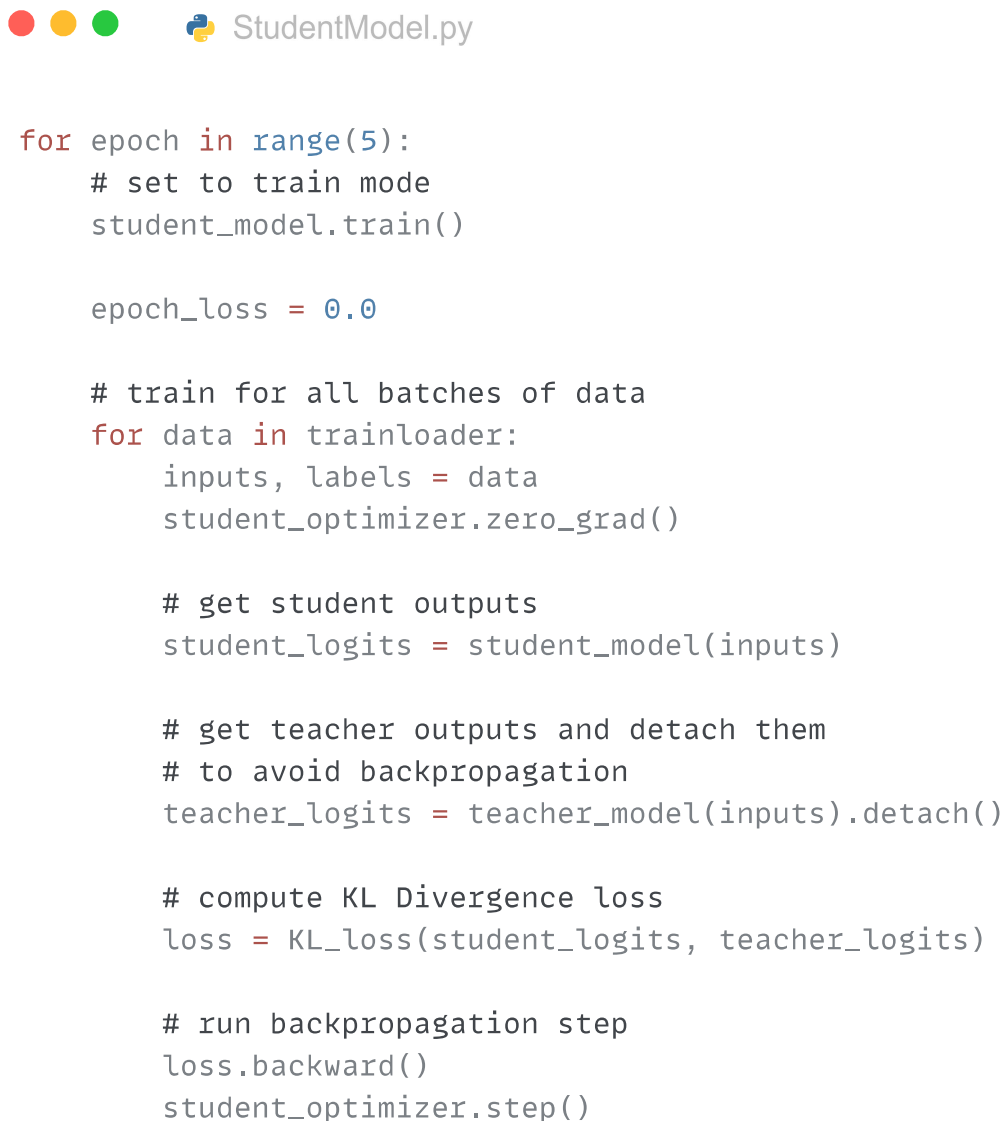
```
StudentModel.py

# Initialize model
student_model = StudentNet()

# Define optimizer
student_optimizer = optim.Adam(student_model.parameters(),
                                lr=0.001)
```

## Step 9) Train the Student model

Finally, it's time to train the Student model.



```
for epoch in range(5):
    # set to train mode
    student_model.train()

    epoch_loss = 0.0

    # train for all batches of data
    for data in trainloader:
        inputs, labels = data
        student_optimizer.zero_grad()

        # get student outputs
        student_logits = student_model(inputs)

        # get teacher outputs and detach them
        # to avoid backpropagation
        teacher_logits = teacher_model(inputs).detach()

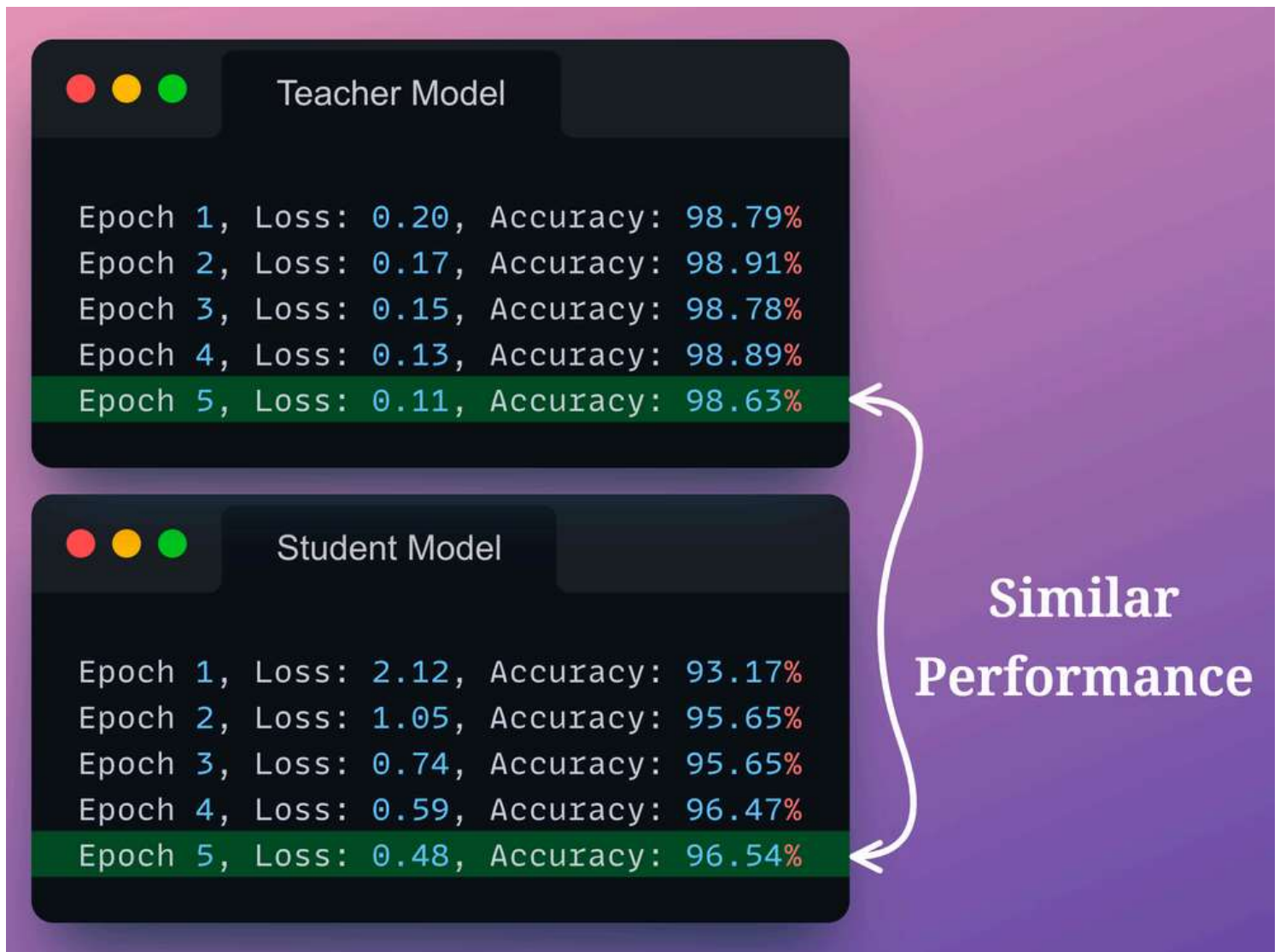
        # compute KL Divergence loss
        loss = KL_loss(student_logits, teacher_logits)

        # run backpropagation step
        loss.backward()
        student_optimizer.step()
```

## Performance comparison — Teacher and Student Model

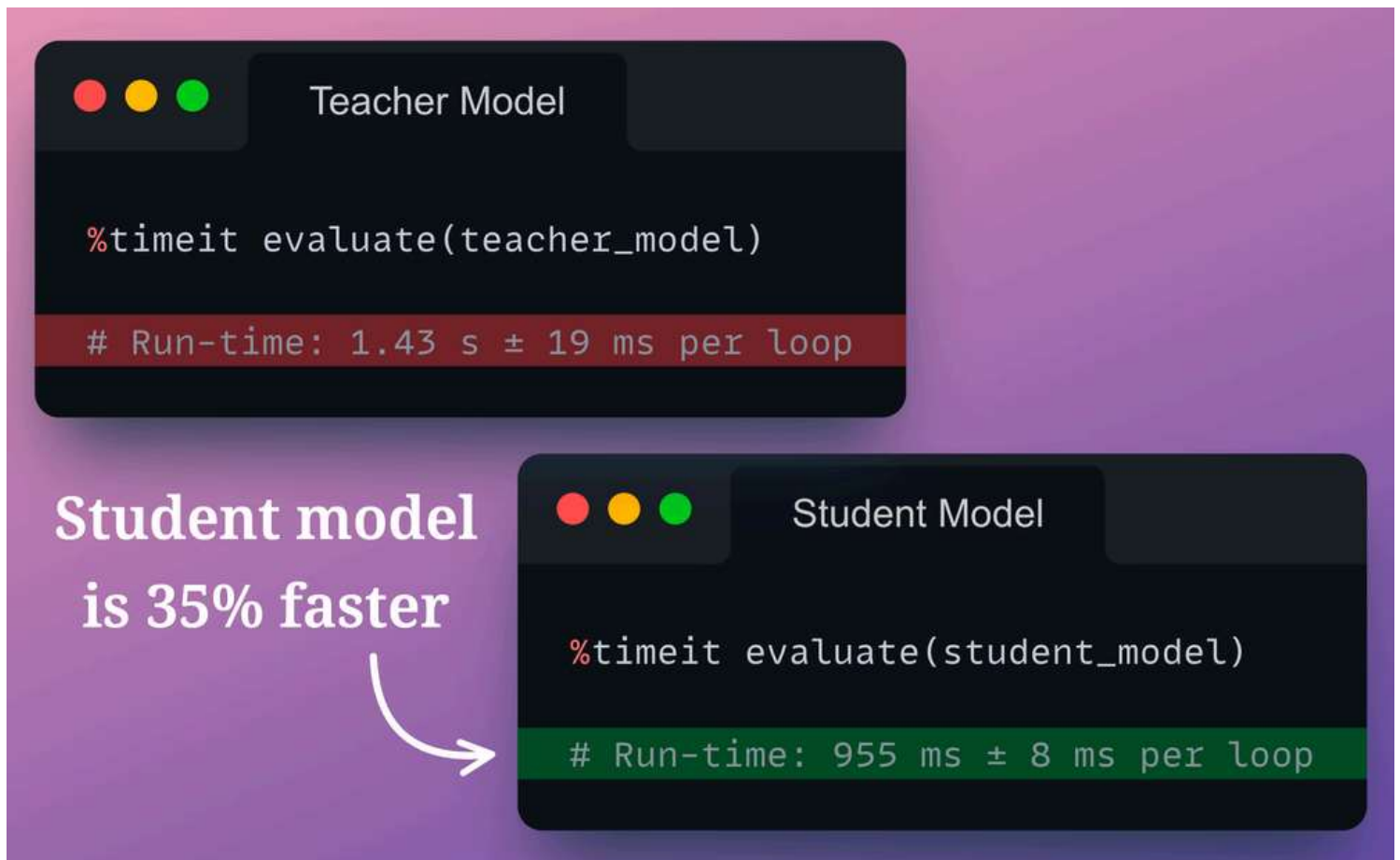
To recap, the teacher model was a CNN-based neural network architecture. The student model, however, was a simple feed-forward neural network.

The following visual compares the performance of the teacher and the student model:



Of course, the performance of the student model is not as good as the teacher model, which is expected. However, it is still pretty close, given that it was only composed of simple feed-forward layers.

Comparing their inference run-time, we get the following:

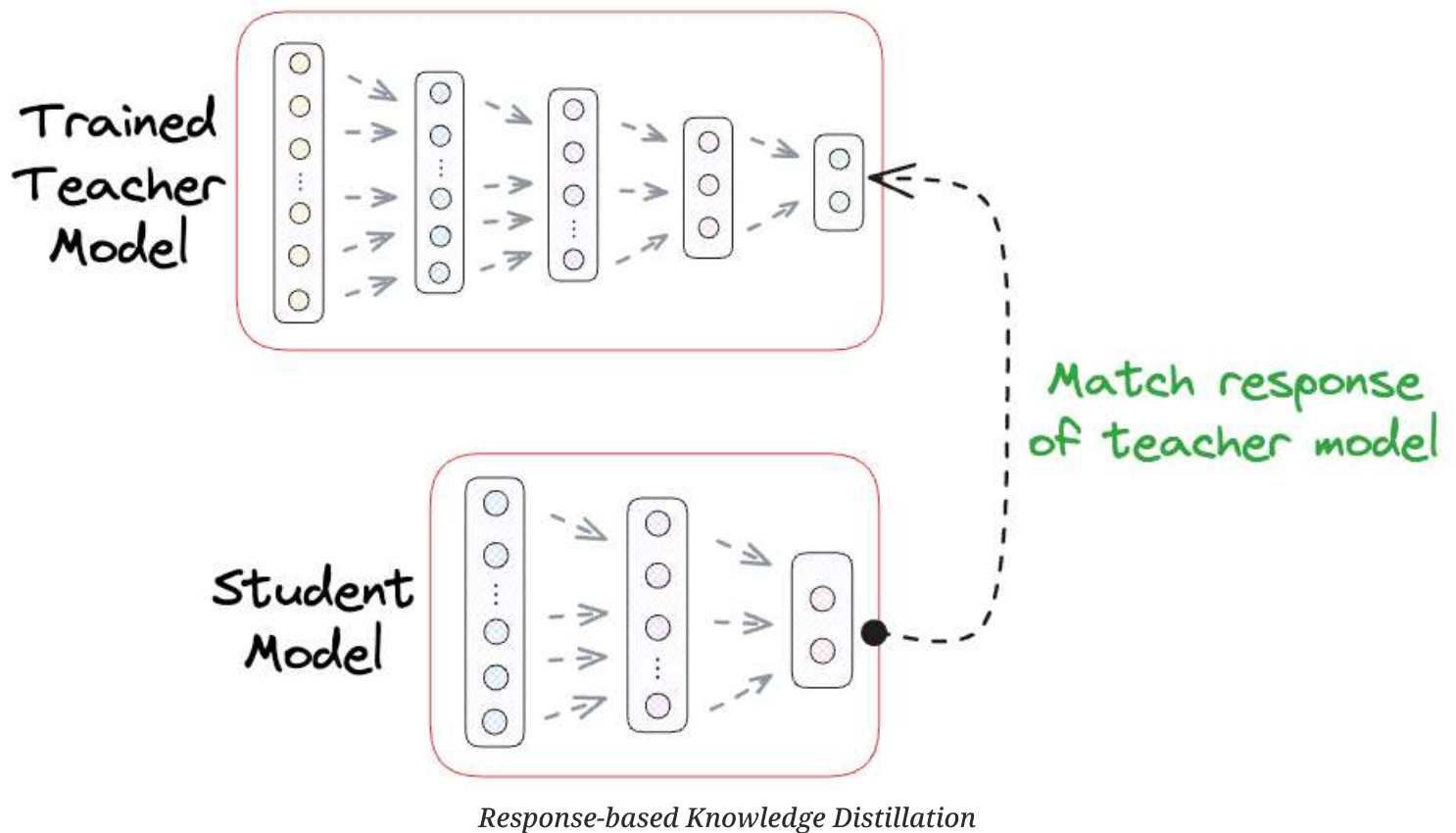


The student model is approximately 35% faster than the teacher model, which is a significant increase in the inference run-time of the model for about a 1-2% drop in the test accuracy.

This is how response-based knowledge distillation works.

To summarize:

- First, we train the large (teacher) model as we typically would.
- Next, we train a smaller (student) model with the objective of mimicking the probability distribution of the class predictions of the teacher model.

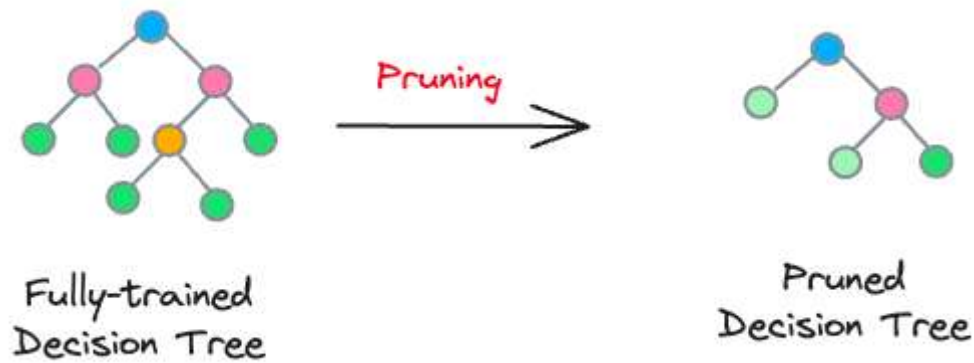


In the above demonstration, we looked at response-based knowledge distillation for a classification use case — **where the response was the class probability**.

However, we can also use response-based knowledge distillation for a regression model. In this case, instead of the KL divergence loss function, we can use a regression-based loss function, mean squared error (MSE), for instance.

## #2) Pruning

Pruning is commonly used in tree-based models, where it involves removing branches (or nodes) to simplify the model.



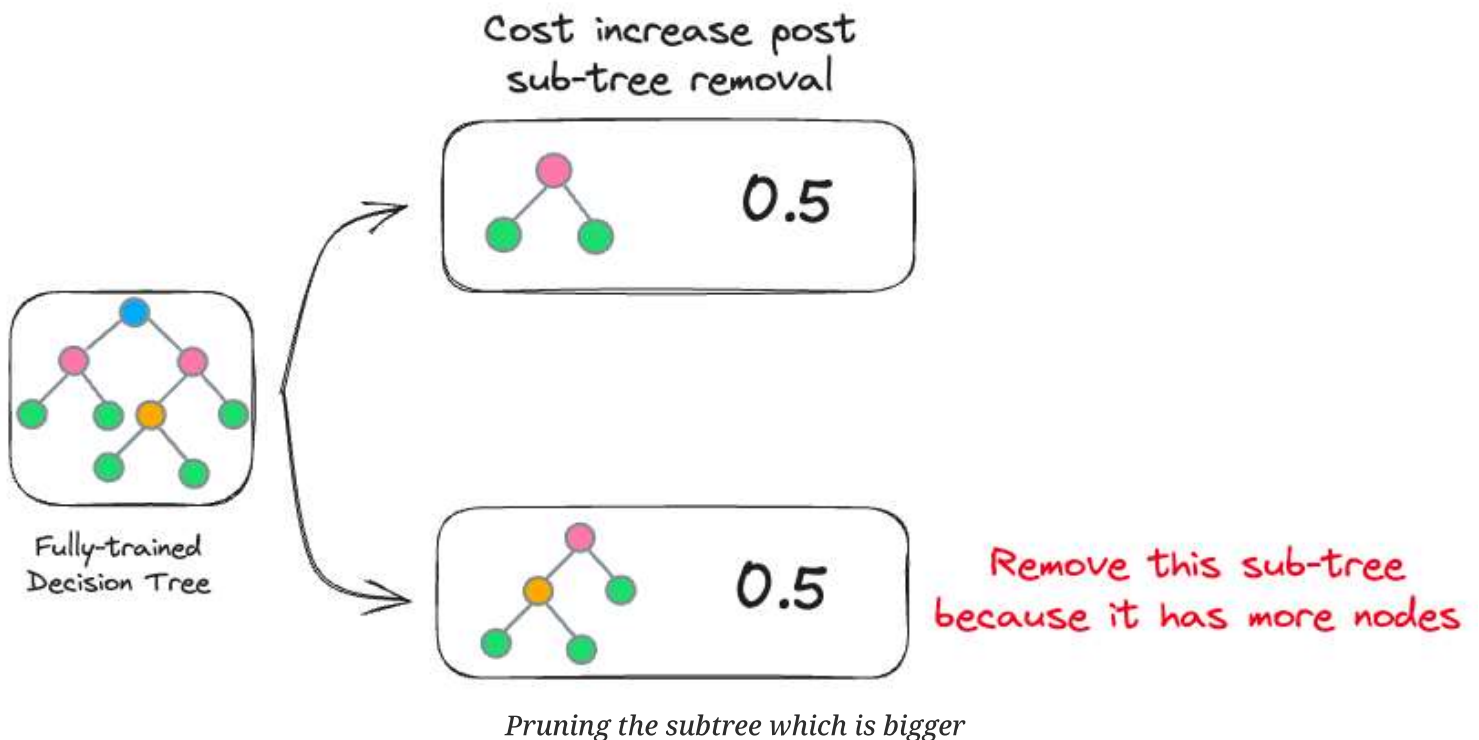
*Pruning a decision tree*

Of course, dropping nodes will result in a drop in the model's accuracy.

Thus, in the case of decision trees, the core idea is to iteratively drop sub-trees, which, after removal, leads to:

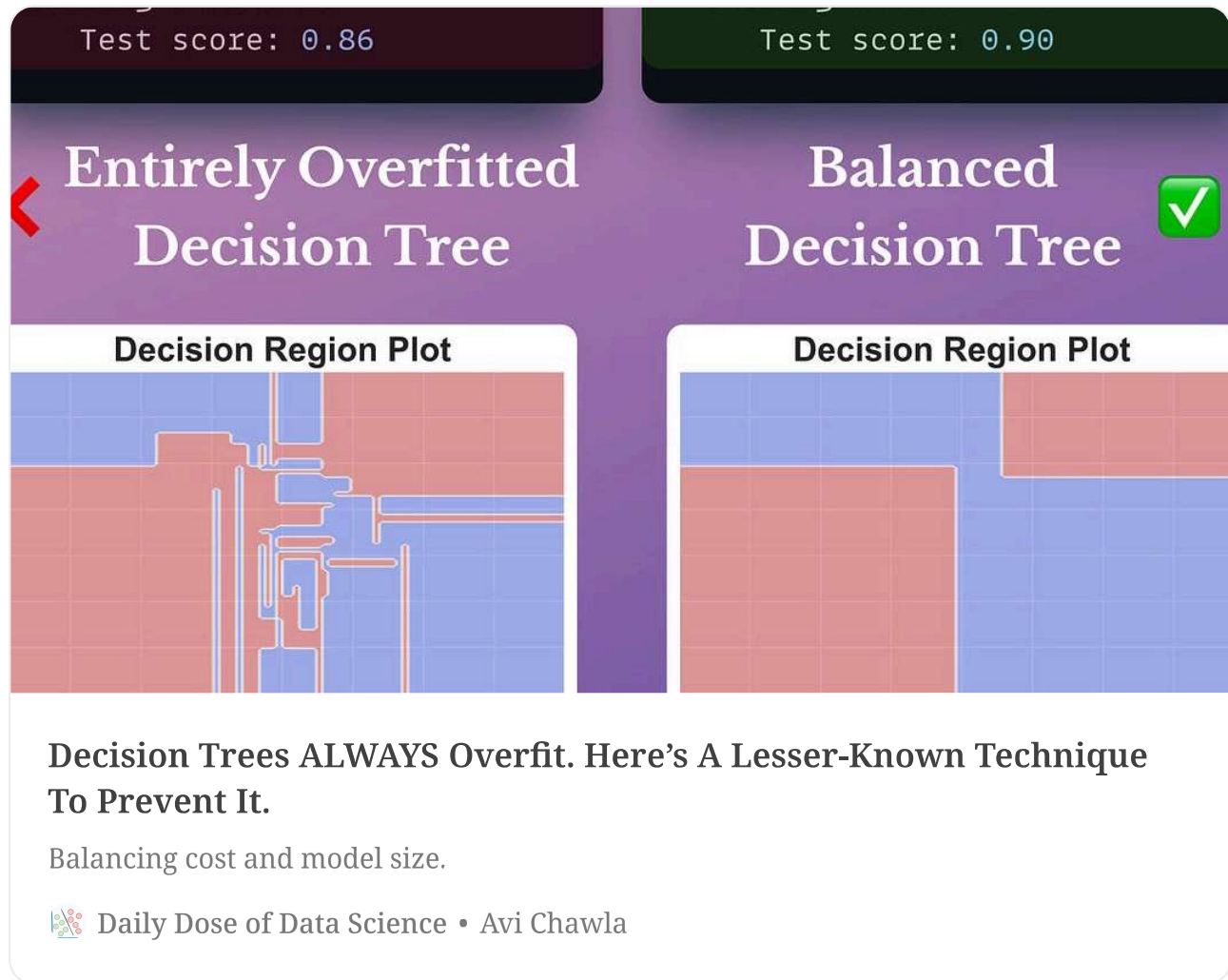
- a minimal increase in classification cost
- a maximum reduction of complexity (or nodes)

This is depicted below:



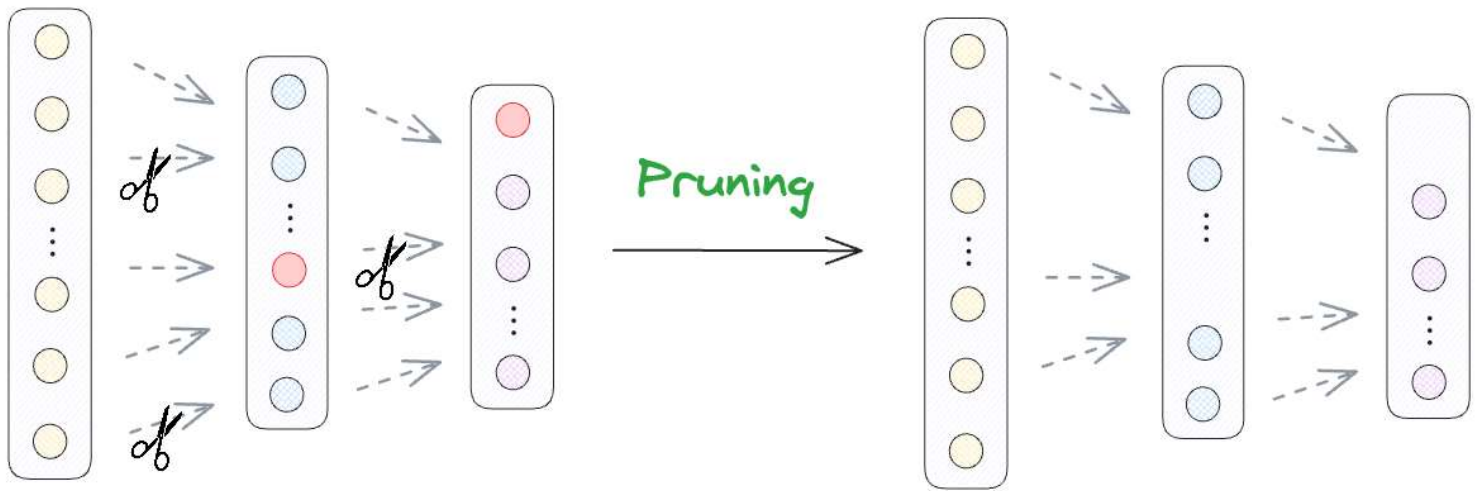
In the image above, both sub-trees result in the same increase in cost. However, it makes more sense to remove the sub-tree with more nodes to reduce computational complexity.

We also discussed this once in the newsletter here:



The same idea can be translated to neural networks as well.

As you may have guessed, pruning in neural networks involves identifying and eliminating specific connections or neurons that contribute minimally to the model's overall performance.



*Pruning in neural networks*

Removing an entire layer is another option. But we rarely practice it because it may result in misaligned weight matrices.

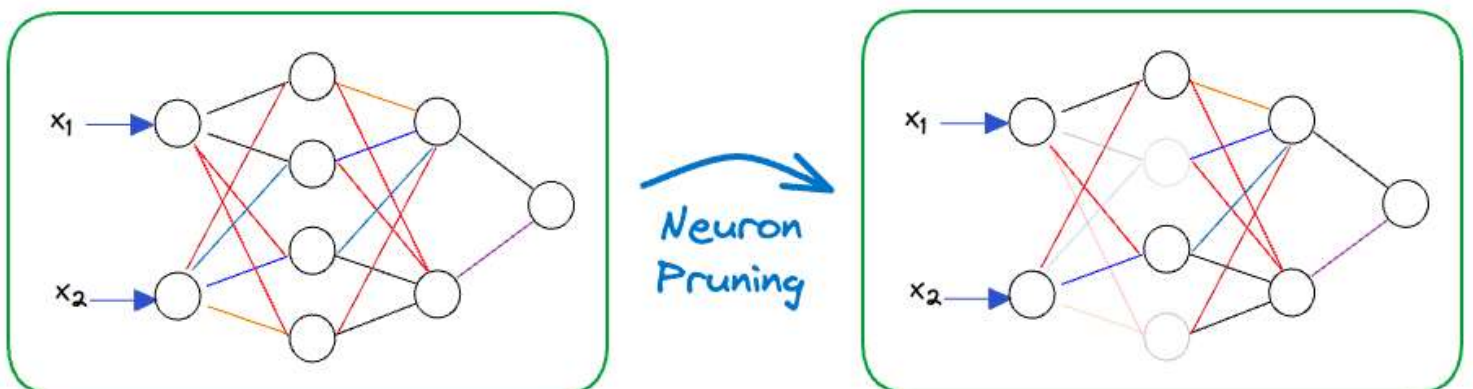
What's more, it isn't easy to quantify the contribution of a specific layer towards the final output.

Thus, instead of removing layers of a trained network, redefining the architecture without the layer we intend to remove can be a better approach.

**With pruning, the goal is to create a more compact neural network while retaining as much predictive power as possible.**

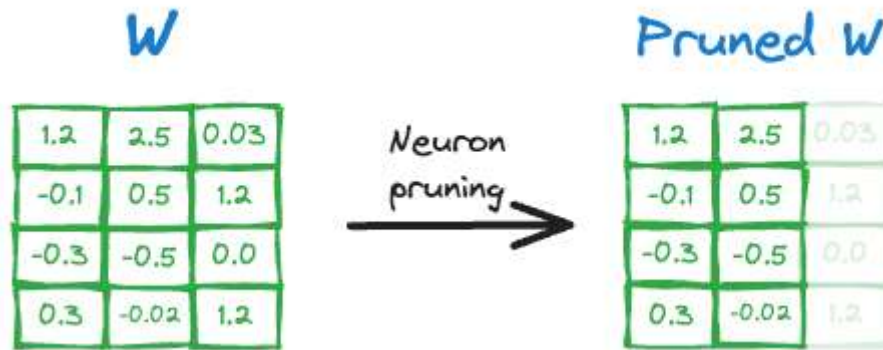
This is primarily done in two ways:

- **Neuron pruning:**



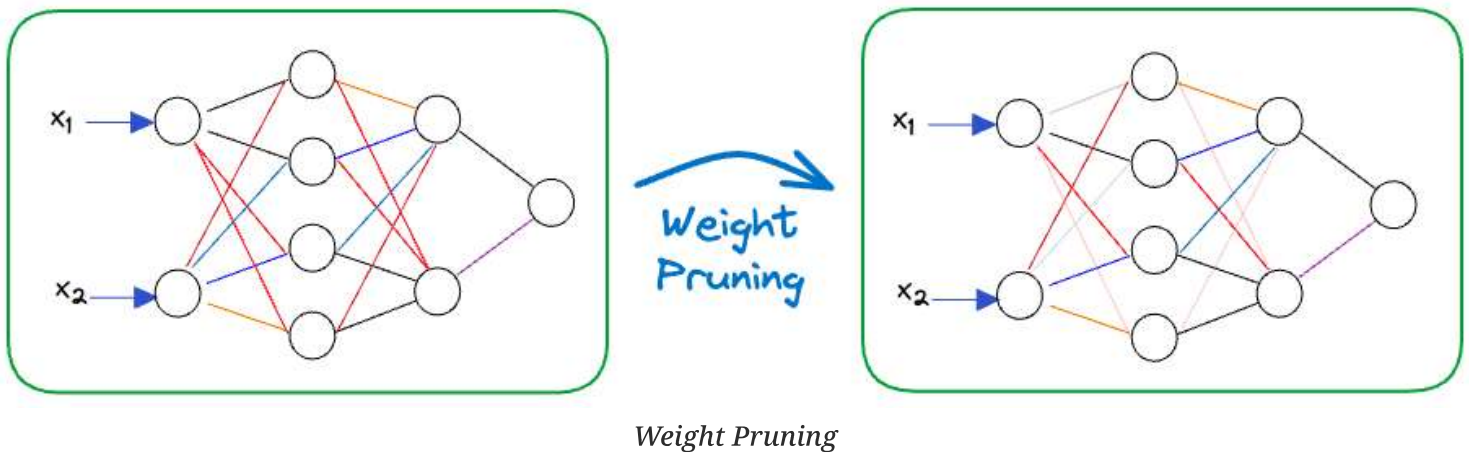
### Neuron Pruning

- The idea is to eliminate entire **nodes** from the network.
- As a result, the matrices representing the layers become small.

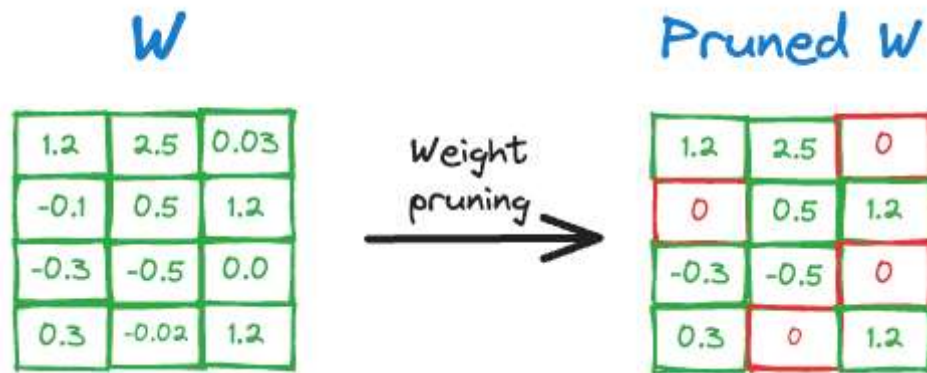


- This results in faster inference and lower memory usage.

- **Weight pruning:**



- This involves eliminating **edges** from the network.
- Weight pruning can be thought of as placing zeros in the matrices to represent the removed edges.



- However, in this case, the size of the matrices remains unaffected.
- Thus, the size of the matrices remains the same, but they become sparse.
- While eliminating edges may not necessarily result in a faster inference, it does help in optimizing memory usage. This is because sparse matrices typically occupy less space than dense matrices (more on this shortly).

💡 We can eliminate both edges and nodes as well.

It is evident from the above ideas that by removing unimportant weights (or nodes) from a network, several improvements can be expected:

- Better generalization
- Improved speed of inference
- Reduced model size

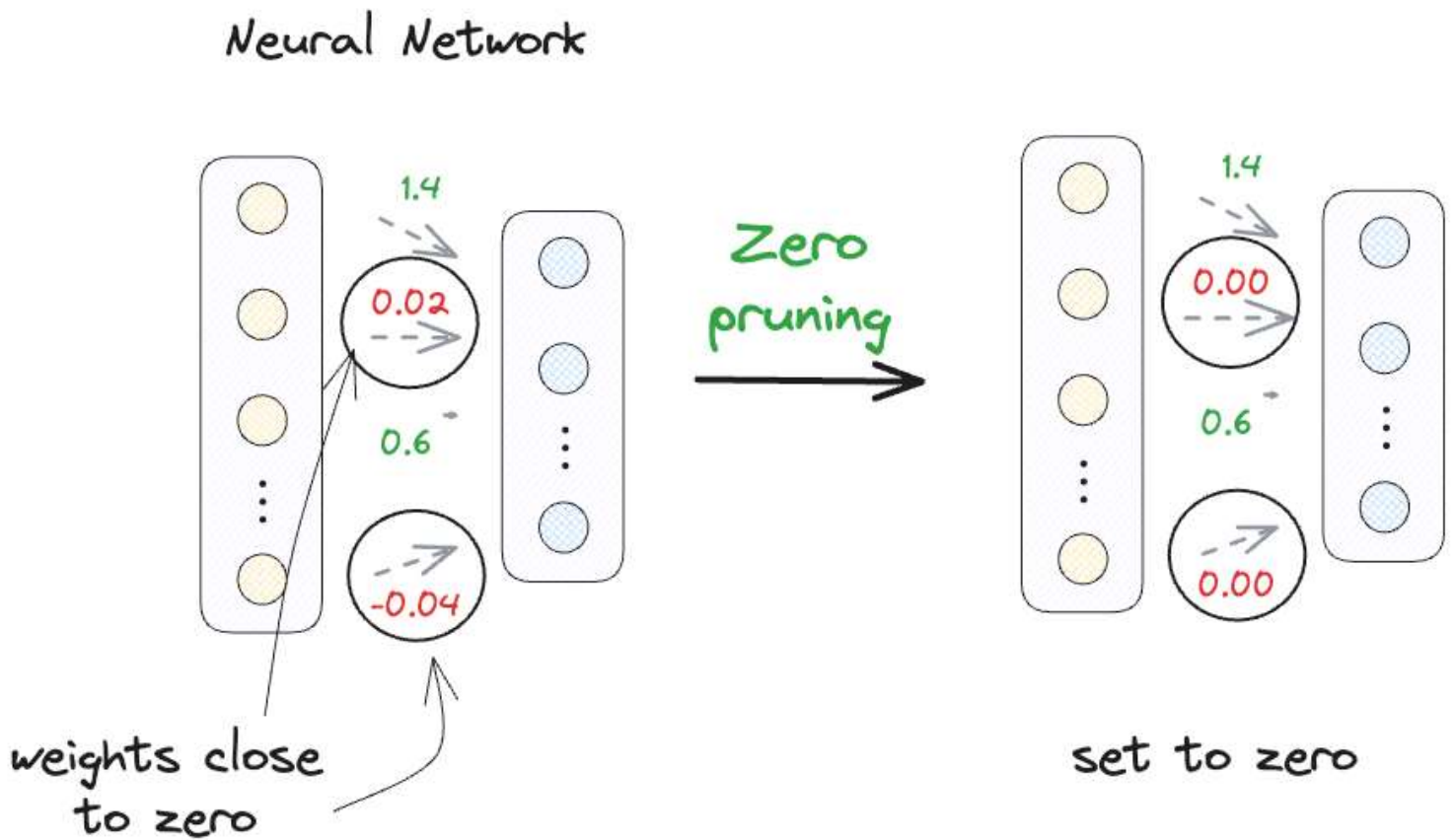
However, the big question is: **Which weights and neurons should we prune?**

There are a few popular ways to implement pruning in neural networks. Let's discuss them.

## #2.1) Zero pruning

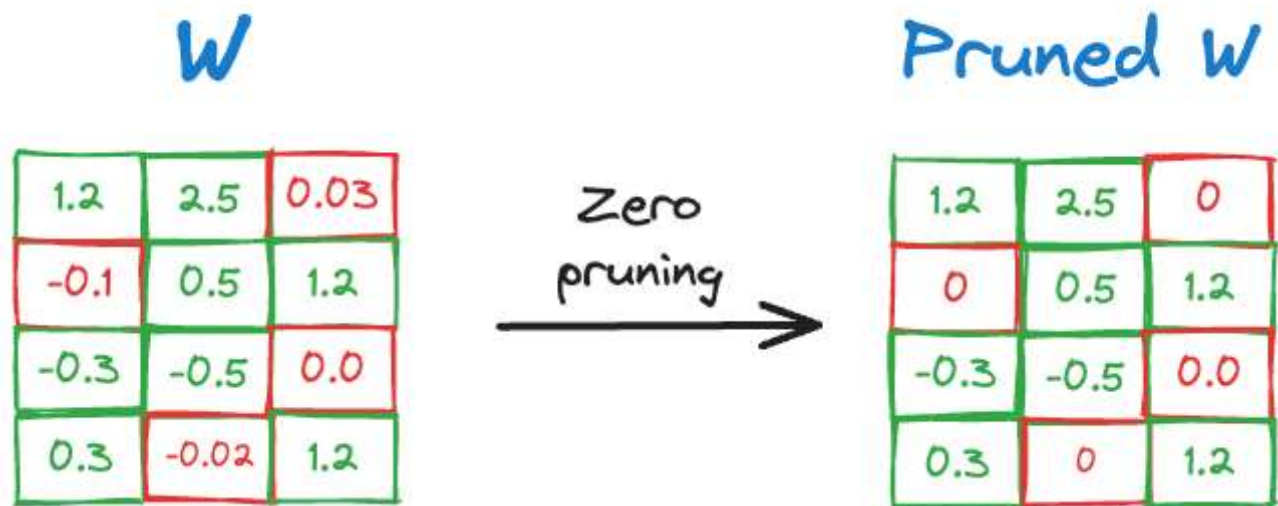
This is one of the simplest ways to prune a neural network, both in terms of understanding and implementation.

Here, the core idea is to remove edges based on the magnitude of the edge weight. Removing an edge means setting its weight to zero.



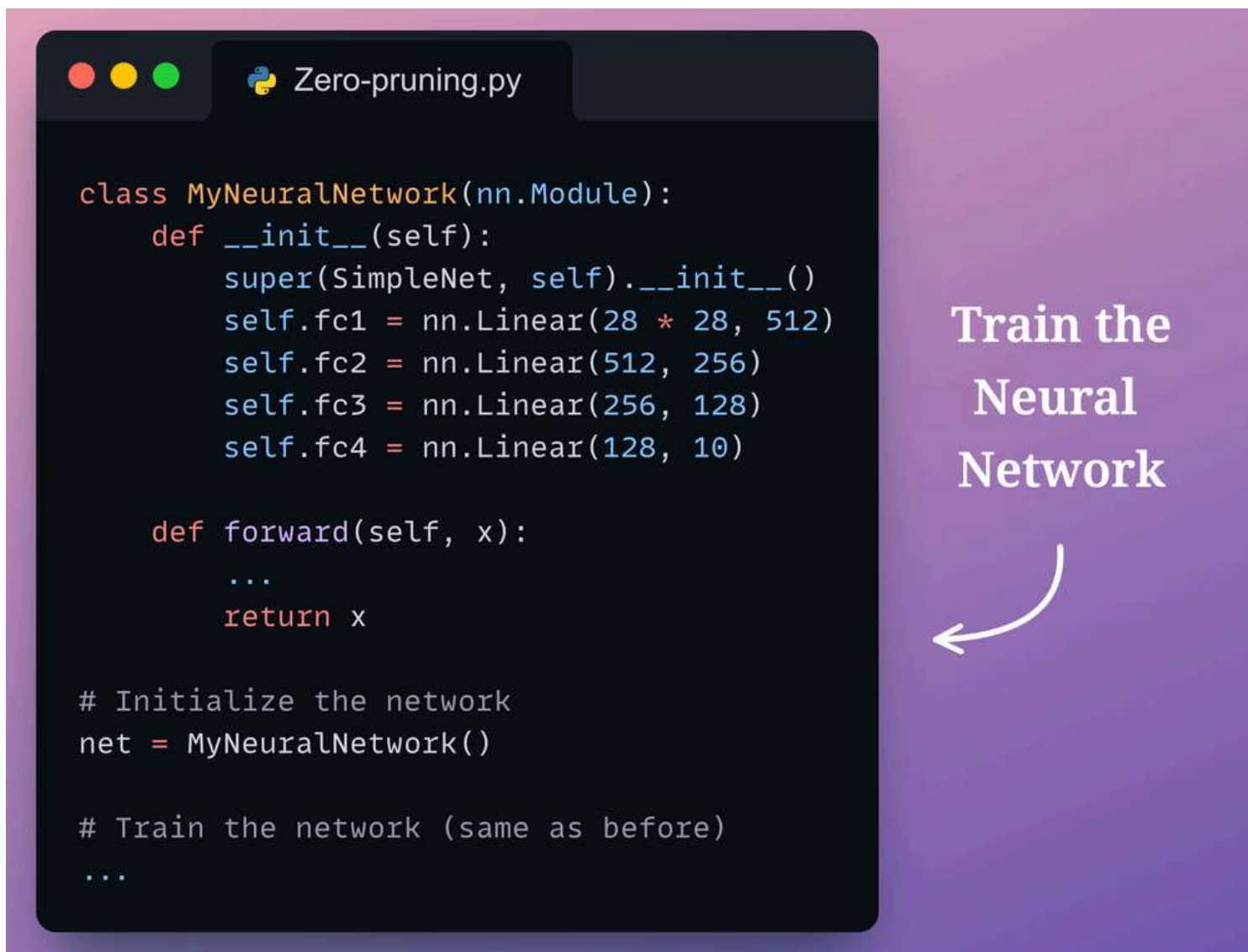
*Zero pruning: Set weights close to zero as zero*

If edge weights are close to zero, this means that they may have a low contribution to the neuron's activation. Thus, we can set these edge weights to zero.



👉 Here's a question: "True or False — Zero pruning is a type of neuron pruning."

Implementation-wise, first, we train the neural network as we usually would. This is depicted below:



Next, we can define a threshold ( $\lambda$ ) for edge weights, say  $\lambda = 0.02$ .

Finally, we zero out all weights below the specified threshold, as demonstrated below:



Zero-pruning.py

```
# Define a pruning threshold (lambda)
threshold = 0.02

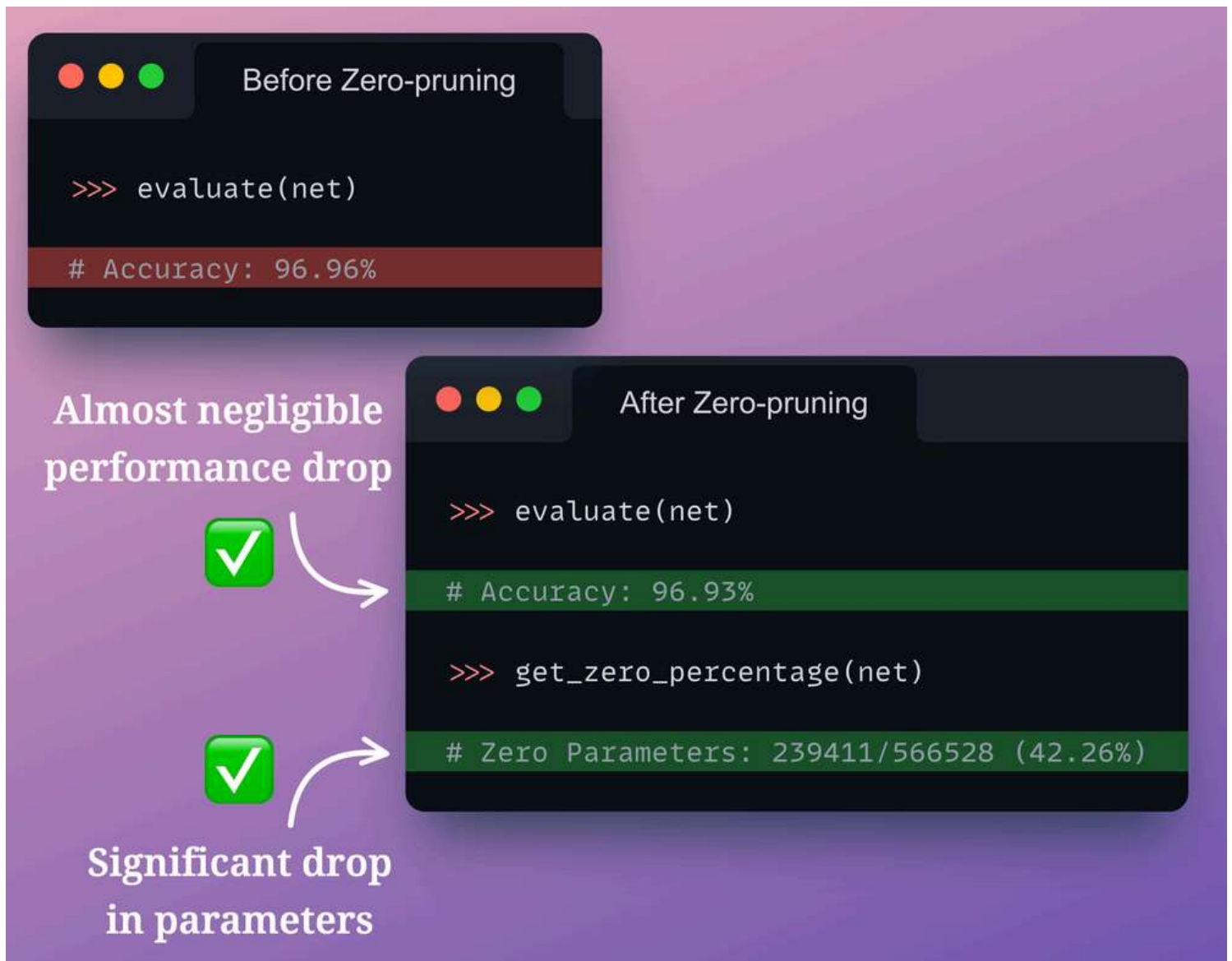
# Implement zero-pruning
for name, param in model.named_parameters():
    if 'weight' in name:
        param.data[param.data.abs() < threshold] = 0.0
```

Set weights  
to zero

As a result, this will zero out all the weights that are below the specified threshold.

Of course, the higher the threshold ( $\lambda$ ), the more will be the zeroed-out weights. **Thus, we must not set a very high threshold as it will zero out many weights, which can degrade the performance.**

Comparing the model's performance before and after zero pruning at  $\lambda = 0.02$ , we get:



As depicted above, on the specific dataset (MNIST), the test accuracy dropped from 96.96% to 96.93%, which is an insignificant drop.

However, a total of 42.26% parameters were zeroed out, which is remarkable.

In reality, however, it is recommended to evaluate the performance across a range of threshold values and subjectively decide the most promising threshold.

For instance, evaluating zero pruning on our model across a range of thresholds gives the following results:

|                   | Threshold | Accuracy | Zero percentage |
|-------------------|-----------|----------|-----------------|
| Original Model    | 0         | 96.96    | -               |
| Zero Pruned Model | 0.01      | 96.93    | 21.36           |
|                   | 0.02      | 96.87    | 42.26           |
|                   | 0.03      | 96.56    | 60.97           |
|                   | 0.04      | 95.00    | 74.46           |
|                   | 0.05      | 89.61    | 82.08           |
|                   | 0.06      | 85.43    | 86.18           |
|                   | 0.07      | 84.97    | 89.07           |
|                   | 0.08      | 79.59    | 91.32           |
|                   | 0.09      | 73.57    | 93.11           |
|                   | 0.1       | 63.62    | 94.50           |

*Results of zero pruning*

The results look pretty impressive, don't they?

Here are a few things to notice:

- At high threshold values (0.08, 0.09, 0.1), there is a significant decrease in the accuracy, which is expected.
- At low threshold values (0.01, 0.02, 0.03), we retain almost the same performance. But at the same time, we zeroed out a significant proportion of parameters.

In this case, the threshold of 0.03 appears to be more promising, resulting in over 60% of zeroed weights.

Given that there are plenty of zero weights, we can represent the weight matrix as a sparse matrix.

Weight  
matrix with  
many zeros



Sparse Matrix

*Represent a matrix with many zeros as a Sparse Matrix*

To efficiently store a sparse matrix with a significant portion of zero parameters in memory, we can use the [Compressed Sparse Row \(CSR\)](#) format.

CSR is a widely used format for sparse matrices because it effectively reduces memory usage.

Moreover, CSR offers efficient arithmetic operations such as matrix addition, multiplication, etc., and fast matrix-vector products, which are essential for inference in neural networks.

To convert the weights matrices to a CSR format, we can use `scipy`.

More specifically, we shall use its `csr_matrix()` method from the `scipy.sparse` package, as demonstrated below:



## Sparse weights

```
# use scipy to sparse matrix
import scipy.sparse as sp

sparse_weights = []

# Convert the pruned weights to a sparse matrix
for name, param in net.named_parameters():
    if 'weight' in name:

        np_weight = param.data.cpu().numpy()
        sparse_weights.append(sp.csr_matrix(np_weight))
```

Convert NumPy array  
to CSR Matrix

Let's also compare the memory usage pre and post-CSR conversion.

First, let's compute the total memory utilized by the zeroed-out weights when they are represented as PyTorch tensors:

```
PyTorch weights

total_size = 0

for name, param in net.named_parameters():
    if 'weight' in name:
        tensor = param.data
        total_size += tensor.element_size() * tensor.numel()

# convert to Megabytes
>>> tensor_size_mb = total_size / (1024**2)
2.16 MBs
```

**Size (in bytes) of each element in a tensor**

**Total elements**

When represented as PyTorch tensors, weights of all layers combined consume 2.16 MBs of memory.

Let's determine the memory usage if we use the CSR format instead.



In contrast to PyTorch tensors, the CSR format takes ~62% less memory, which is a massive decrease in memory utilization.



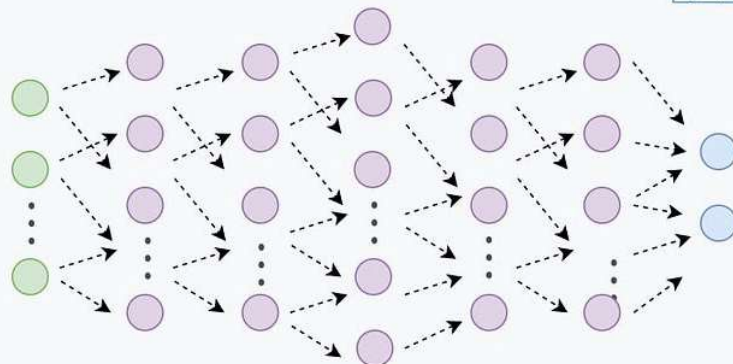
If you want to stick to PyTorch tensors and you can compromise a bit on the overall run-time, then **Gradient Checkpointing** is a reliable technique to **train** large neural networks. Do note that this technique is mainly used to optimize training, not inference. We discussed it below if you are interested in learning more.

## Gradient Checkpointing: Save 50-60% Memory When Training a Neural Net

**p 1) Run the forward pass normally**



blog.DailyDoseofDS.



**p 2) Divide the network into segments before backpropagation**

### Gradient Checkpointing: Save 50-60% Memory When Training a Neural Network

An underrated technique to train larger ML models.

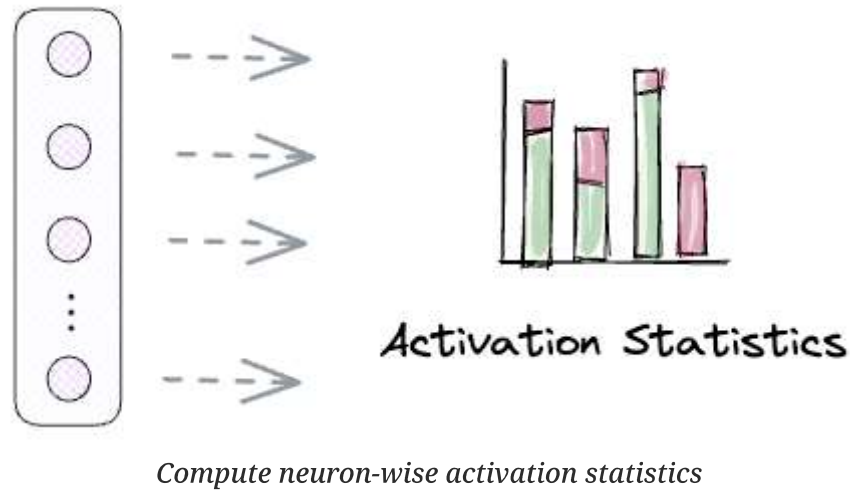


Daily Dose of Data Science • Avi Chawla

## #2.2) Activation pruning

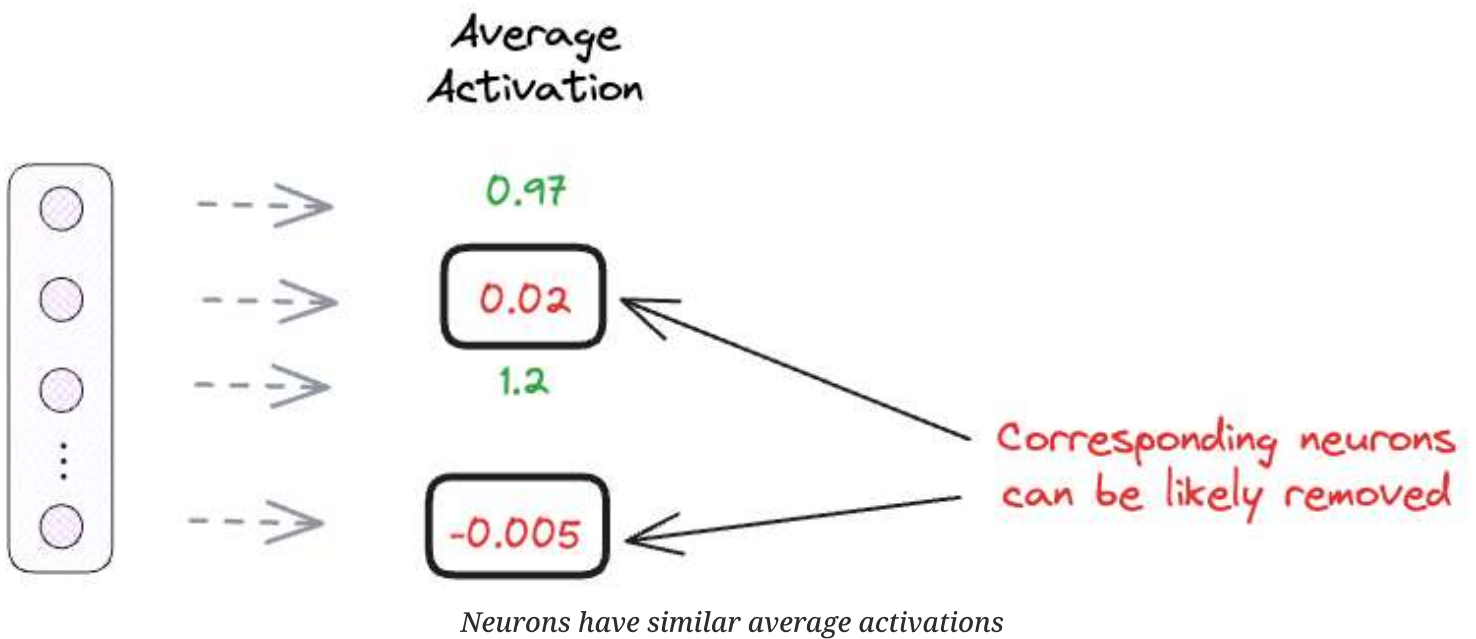
Rather than solely relying on threshold-based zero pruning, neuron activations can also be used as a criterion for pruning.

Essentially, when running a dataset through a network, we can compute and analyze certain statistics of the individual neuron activations.



Here, we may observe that some neurons always output near-zero values.

As a result, they can likely be removed as they will have little impact on the model's output.



This makes intuitive sense as well — if a neuron rarely possesses high activation value, then it is fair to assume that it isn't contributing to the model's output.

👉 Here's a question: "True or False — Activation-pruning is a type of neuron pruning."

Here are the steps we need to follow to implement activation pruning:

- Train the neural network as we typically would.
- After training, run the training data again through the network (without backpropagating the gradients).
- Compute the average activations of every neuron in the hidden layers across the entire data.
- Define a pruning threshold  $\lambda$ .
- Flag neurons with activations below the pruning threshold  $\lambda$ .
- Remove the flagged neurons from the weight matrices of the trained network.
- Done!

Let's implement it now.

The process for training the initial neural network will remain the same, so we shall not discuss it again.

To recap, we are dealing with the following neural network architecture. This will be helpful in computing the intermediate activations ahead.

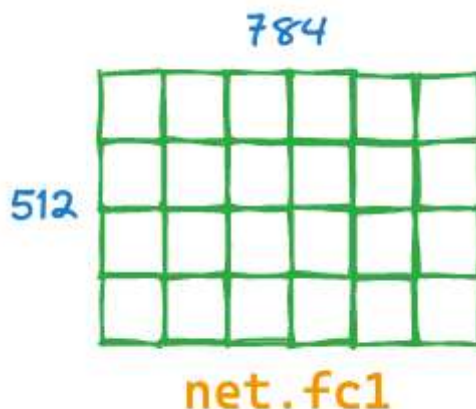
```
class MyNeuralNetwork(nn.Module):  
    def __init__(self):  
        super(SimpleNet, self).__init__()  
        self.fc1 = nn.Linear(28 * 28, 512)  
        self.fc2 = nn.Linear(512, 256)  
        self.fc3 = nn.Linear(256, 128)  
        self.fc4 = nn.Linear(128, 10)  
  
    def forward(self, x):  
        ...  
        return x  
  
# Initialize the network  
net = MyNeuralNetwork()  
  
# Train the network (same as before)  
...
```

Train the  
Neural  
Network

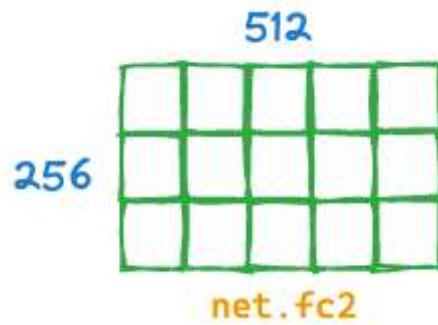


Essentially, we have four layers in this network:

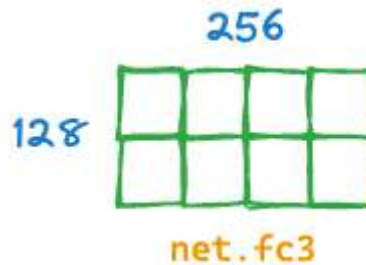
- `net.fc1`: The input layer.
  - It transforms the 784 dimensional input to a 512 dimensional latent representation.



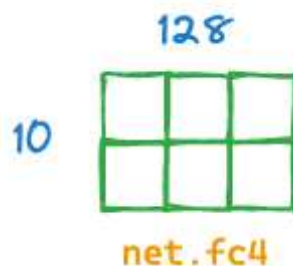
- Thus, `net.fc1` will be matrix of shape (512, 784).
- `net.fc2`: The first hidden layer
  - This layer transforms the 512 dimensional output of the input layer to 256 dimensional latent representation.



- Thus, `net.fc2` will be matrix of shape (256, 512).
- `net.fc3`: The second hidden layer.
  - This layer transforms the 256 dimensional output of the first hidden layer to 128 dimensional latent representation.



- Thus, `net.fc3` will be matrix of shape (128, 256).
- `net.fc4`: The output layer.
  - This layer takes the output of the second hidden layer and outputs the class-wise logits.



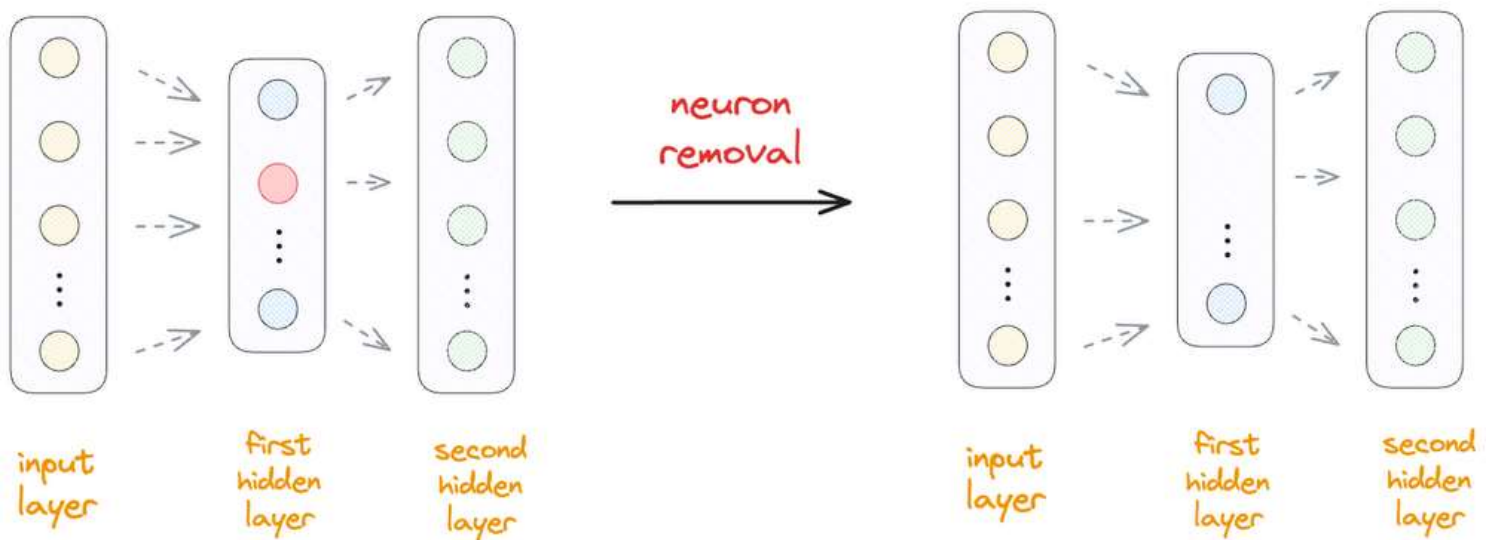
- Thus, `net.fc4` will be matrix of shape (10, 128).

As discussed earlier, the objective is to **remove neurons** whose average activations are below the specified threshold  $\lambda$ .

💡 We must only remove neurons from the hidden layers because that is precisely where activations are computed. Also, if we remove neurons from the input layer or the output layer, it will conflict with the desired input and output shape.

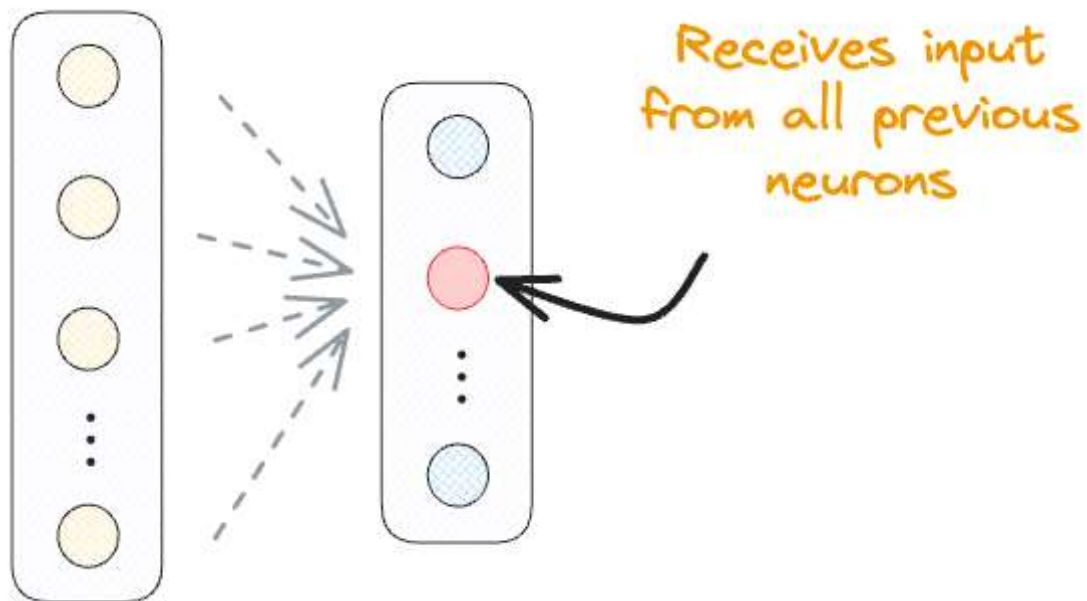
Removing neurons means that we shall be affecting the size of the layers in our network.

For instance, say we remove the red neuron in the first hidden layer of the following network. As a result, the number of neurons in that layer will decrease.



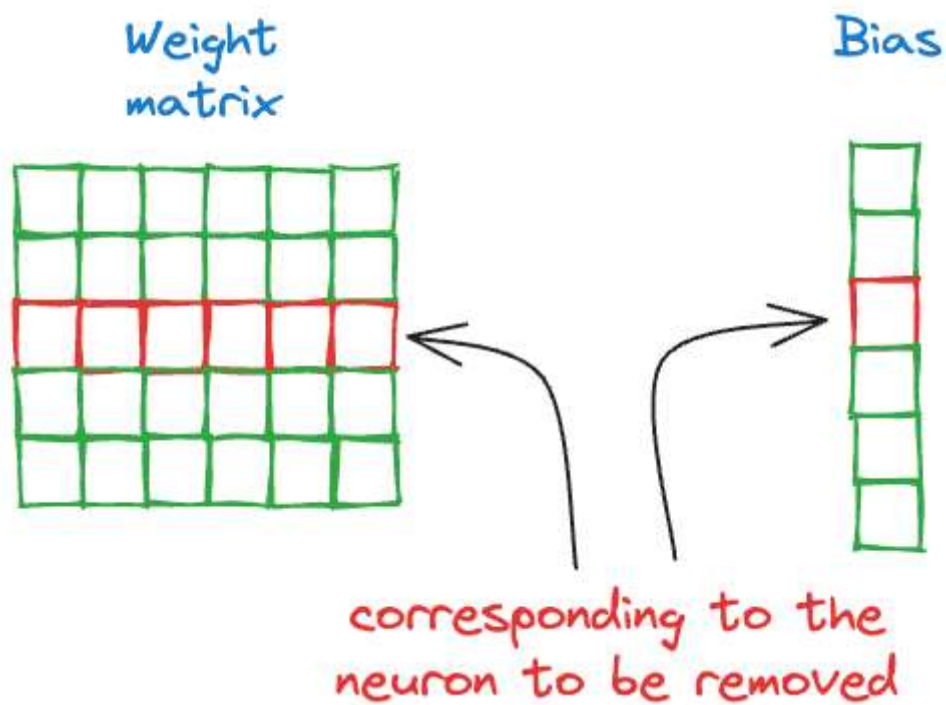
Let's see how removing a neuron will affect the weight matrices.

A neuron in any hidden layer receives its input from all neurons of the previous layer.

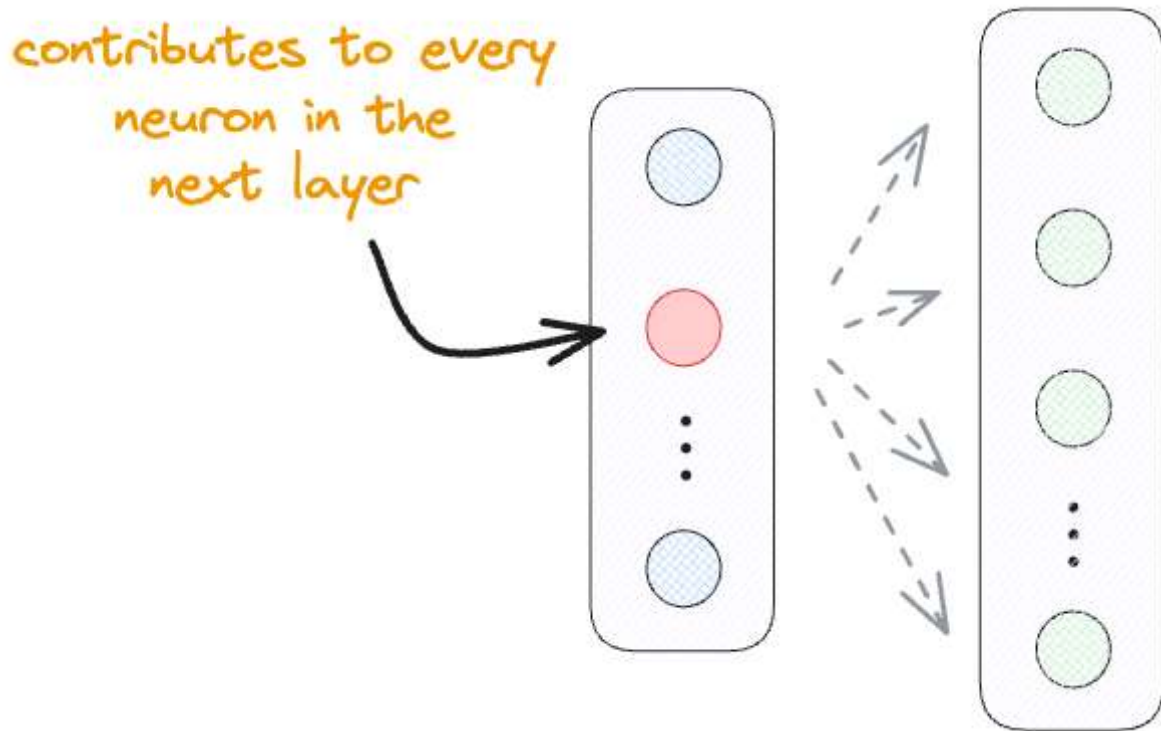


*Neuron receives input from all previous neurons*

Thus, removing a neuron would mean that we should remove the entire row from the weight matrix that generates the input for that neuron:

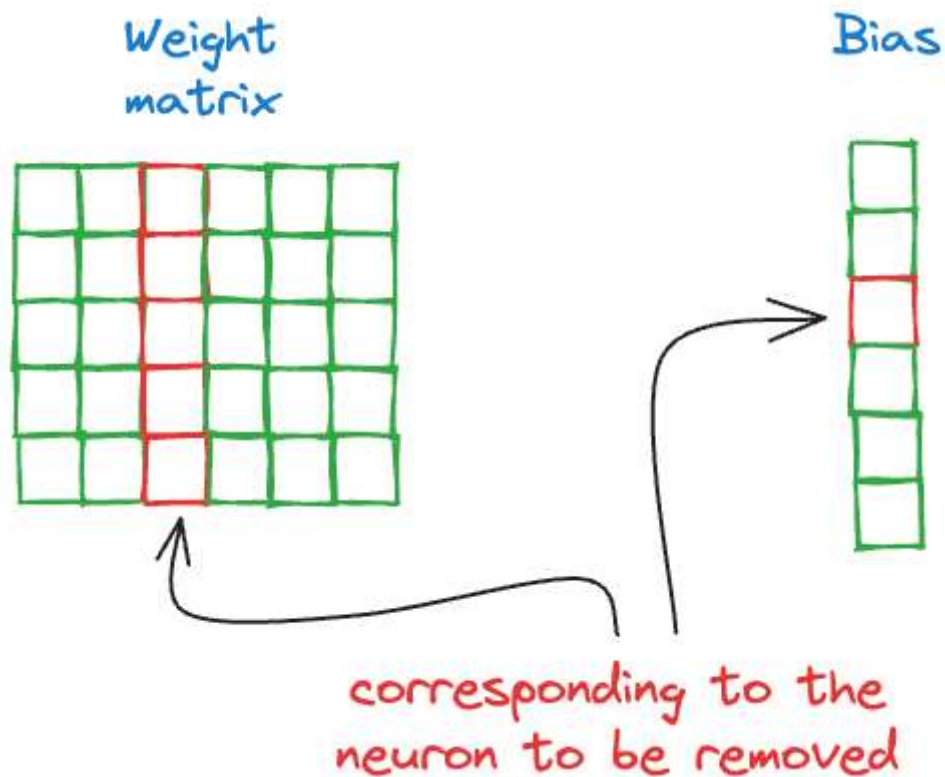


Furthermore, a neuron in any hidden layer also contributes towards the computation of every neuron in the next layer:



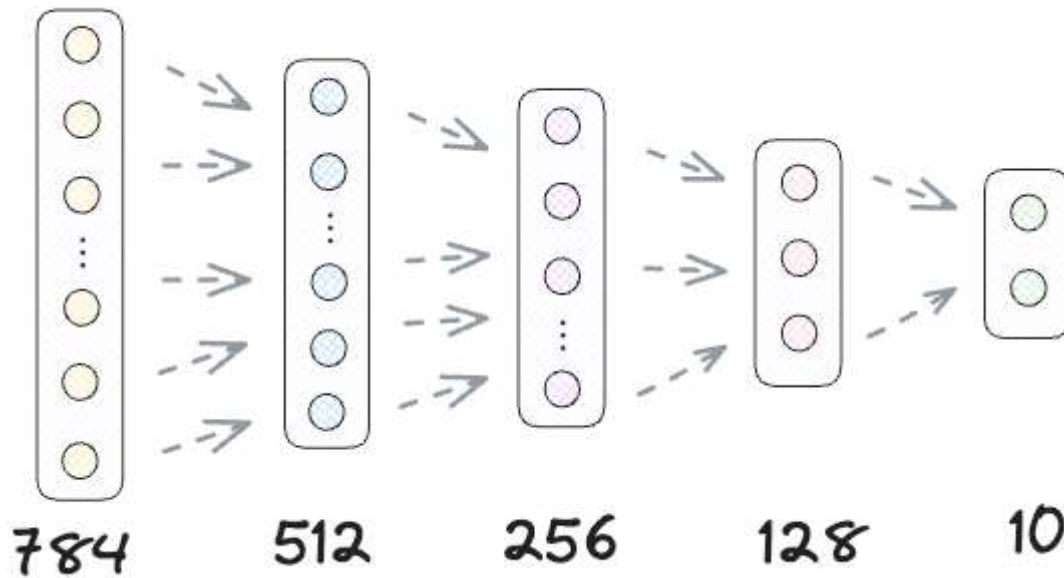
*Neuron contributes to all neurons in the next layer*

Thus, removing this neuron would mean that we must remove the entire column from the weight matrix that connects this neuron to the next layer:



With this in mind, let's proceed with implementing activation pruning.

The architecture of our neural network is depicted below:



As discussed above, we must only store the average activations corresponding to the hidden layers in the network — three in this case.

Thus, we first define a list of size three to store them:

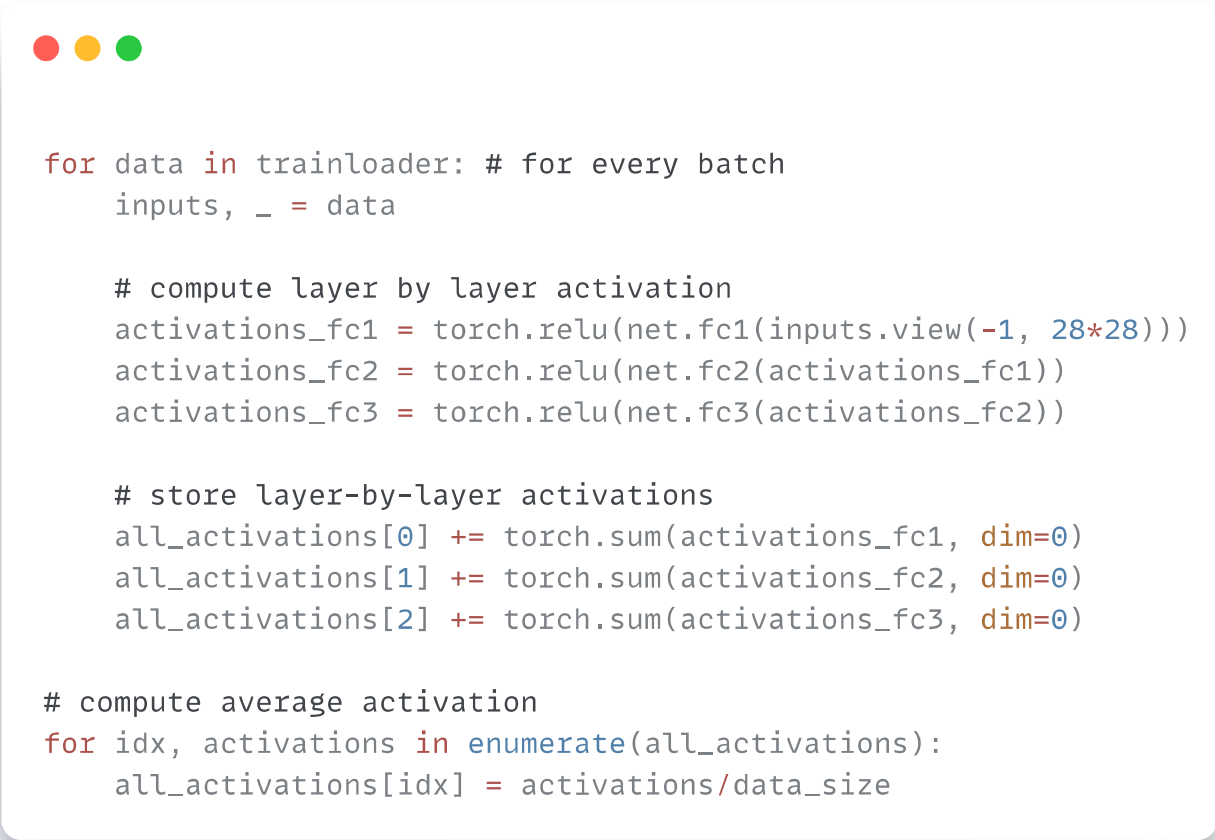
```
all_activations = [torch.zeros(512), # 1st hidden layer  
                   torch.zeros(256), # 2nd hidden layer  
                   torch.zeros(128)  # 3rd hidden layer  
                   ]
```

Copy

Next, we will do the following:

- For every batch of data:
  - Compute the activations of each hidden layer in the network.
  - Sum the activations to get neuron-wise activations of each hidden layer.
  - Store them in the corresponding tensors of `all_activations` list.
- Compute the average activation of each neuron in the `all_activations` list.

This is depicted below:



```
for data in trainloader: # for every batch
    inputs, _ = data

    # compute layer by layer activation
    activations_fc1 = torch.relu(net.fc1(inputs.view(-1, 28*28)))
    activations_fc2 = torch.relu(net.fc2(activations_fc1))
    activations_fc3 = torch.relu(net.fc3(activations_fc2))

    # store layer-by-layer activations
    all_activations[0] += torch.sum(activations_fc1, dim=0)
    all_activations[1] += torch.sum(activations_fc2, dim=0)
    all_activations[2] += torch.sum(activations_fc3, dim=0)

# compute average activation
for idx, activations in enumerate(all_activations):
    all_activations[idx] = activations/data_size
```

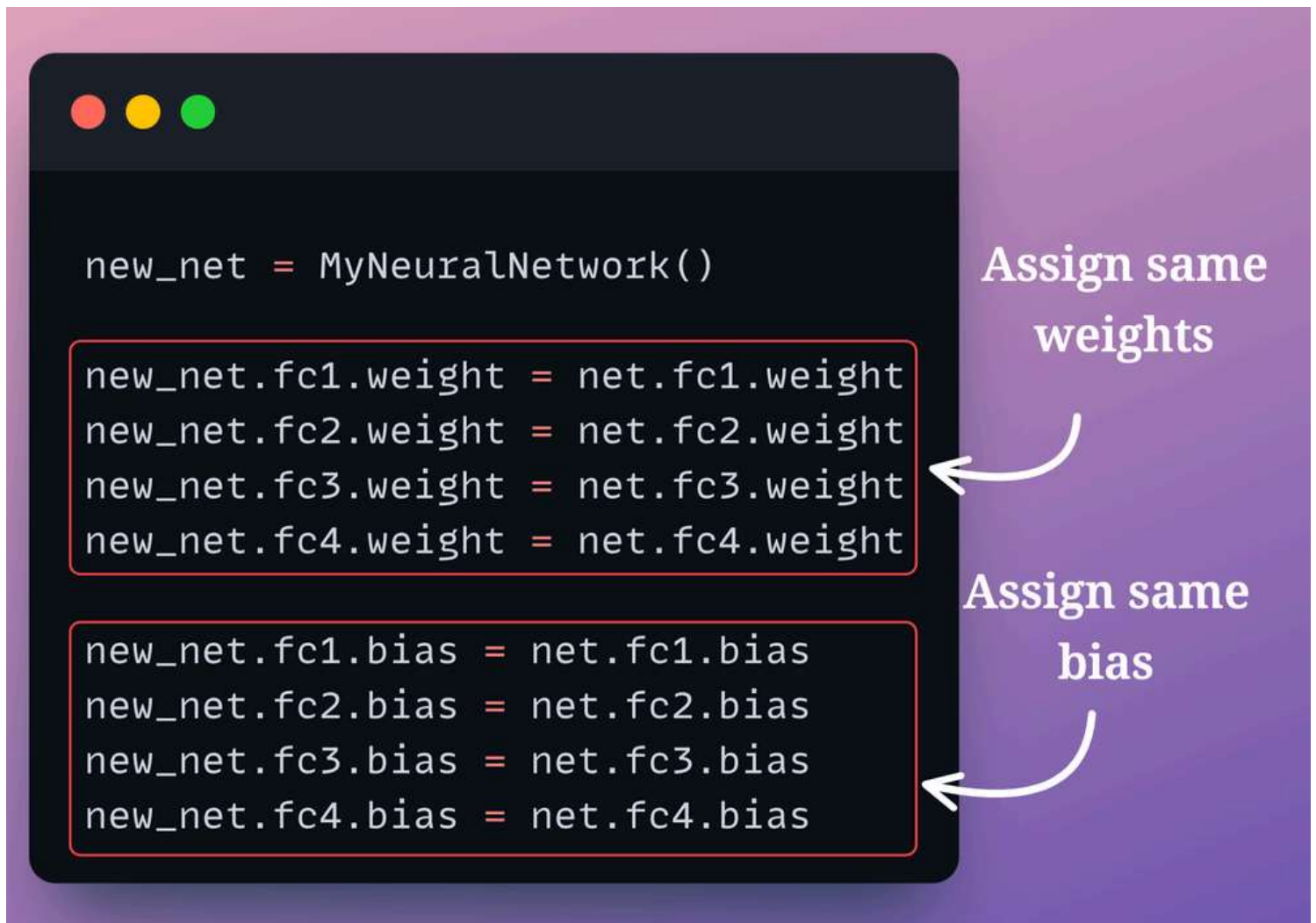
And done!

Now, we shall use these neuron-specific average activations computed across the entire dataset to determine which neurons have low activations on average.

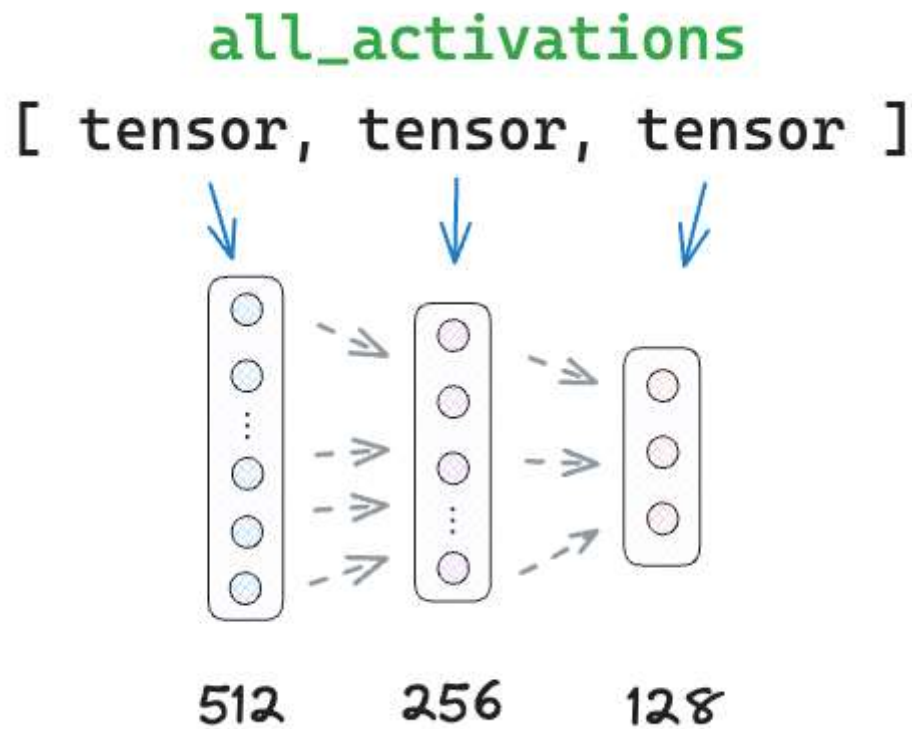
👉 At times, determining the average activations across the entire training can be difficult, especially when the training set is large. In such cases, you may use a sample of the training data.

If the average activation is less than the specified threshold, we will remove the neuron from the weight matrices it is associated with.

First, we instantiate a new neural network and assign it the weights of the trained neural network:



Next, we shall take the average activation for every neuron in the hidden layer from the list `all_activations` and remove neurons from the weight matrices of the network `new_net` by applying thresholds on the average activations.



*Store activations of all neurons in the hidden layer*

For instance, to remove neurons from the first hidden layer, we will modify the weights of `new_net.fc1` and `new_new.fc2`.

👉 We discussed why two matrices should be modified to remove a neuron above.

Furthermore, we shall also filter out the corresponding bias term of the removed neuron.

These updates are demonstrated below:



```
# apply threshold on first hidden layer
keep_neurons_l1 = all_activations[0]>threshold

# remove rows (neurons) that have activations below threshold
new_net.fc1.weight = nn.Parameter(new_net.fc1.weight[keep_neurons_l1])

# remove columns (neurons) that have activations below threshold
new_net.fc2.weight = nn.Parameter(new_net.fc2.weight[:, keep_neurons_l1])

# remove bias of neurons that have activations below threshold
new_net.fc1.bias = nn.Parameter(new_net.fc1.bias[keep_neurons_l1])
```

We must do the same for all hidden layers.



Remove neurons

```
## Second hidden layer
keep_neurons_l2 = all_activations[1]>threshold

new_net.fc2.weight = nn.Parameter(new_net.fc2.weight[keep_neurons_l2])
new_net.fc3.weight = nn.Parameter(new_net.fc3.weight[:, keep_neurons_l2])

new_net.fc2.bias = nn.Parameter(new_net.fc2.bias[keep_neurons_l2])

## Third hidden layer
keep_neurons_l3 = all_activations[2]>threshold

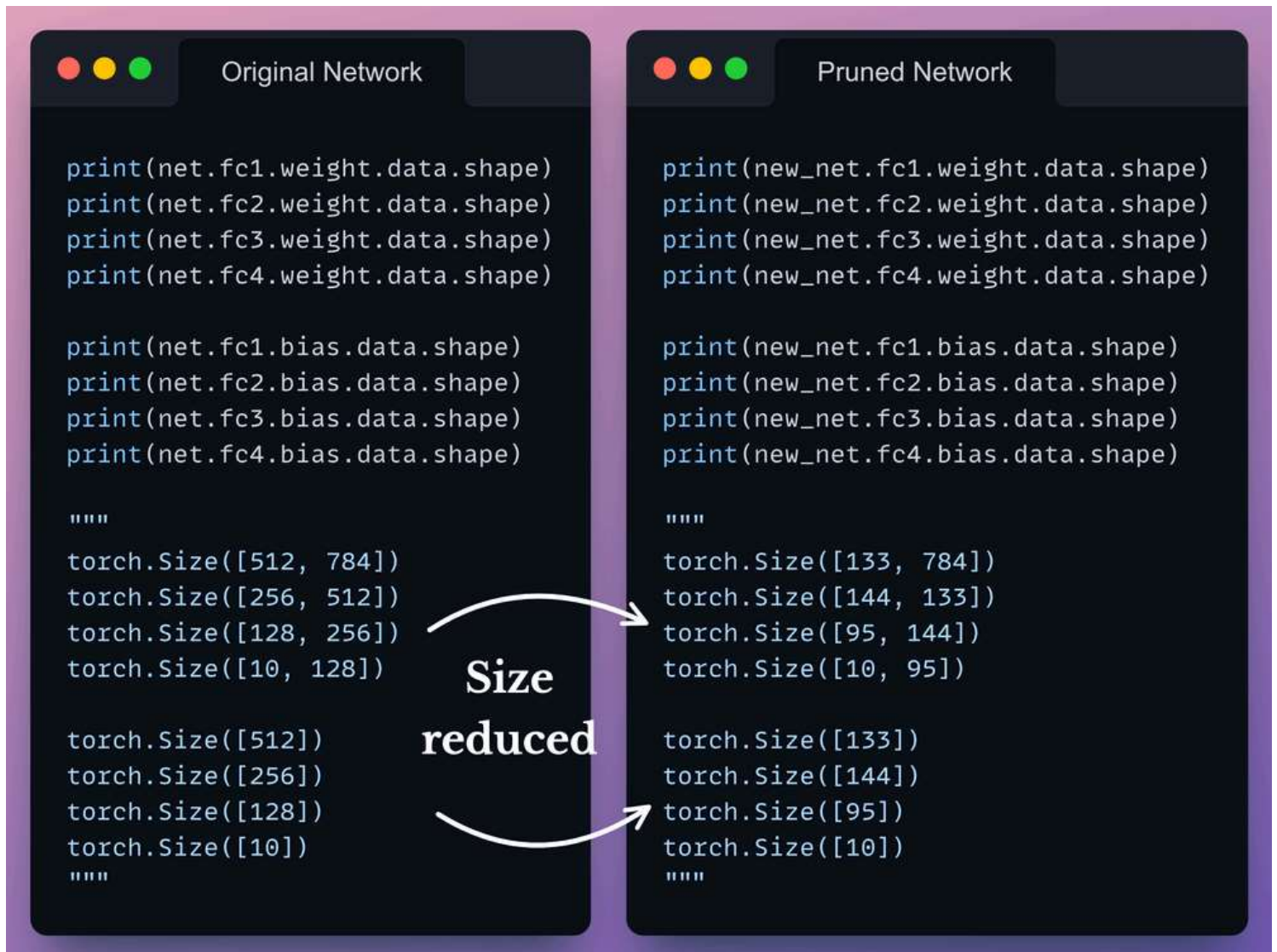
new_net.fc3.weight = nn.Parameter(new_net.fc3.weight[keep_neurons_l3])
new_net.fc4.weight = nn.Parameter(new_net.fc4.weight[:, keep_neurons_l3])

new_net.fc3.bias = nn.Parameter(new_net.fc3.bias[keep_neurons_l3])
```

Done.

With this, we have eliminated all neurons with average activations below the specified threshold.

We can also verify the change in the shape of matrices:



As depicted above, we have significantly reduced the shape of the weight matrices by removing neurons with average activations below the specified threshold.

But now is the real test. In other words, we must answer the question: **“How effective was activation pruning?”**

Instead of evaluating the performance on a specific threshold, let’s assess the pruned network across a range of thresholds.

We shall look at the following metrics:

- Accuracy of the pruned network.

- Number of parameters in the pruned network.
- Inference time.

The results are shown below, and they look pretty impressive:

|                | Threshold | Accuracy | Inference Time | Size Reduction |
|----------------|-----------|----------|----------------|----------------|
| Original Model | 0         | 96.69    | 1.18           | 0%             |
| Pruned Model   | 0.1       | 96.62    | 1.09           | 62%            |
|                | 0.2       | 96.41    | 1.01           | 66%            |
|                | 0.3       | 96.31    | 1.28           | 69%            |
|                | 0.4       | 96.08    | 1.19           | 72%            |
|                | 0.5       | 94.88    | 1.10           | 74%            |
|                | 0.6       | 94.49    | 1.08           | 76%            |
|                | 0.7       | 92.19    | 1.22           | 79%            |
|                | 0.8       | 90.65    | 1.30           | 81%            |
|                | 0.9       | 88.39    | 1.21           | 83%            |
|                | 1         | 87.87    | 1.30           | 84%            |

*Results of activation pruning*

At a threshold of just 0.1, we reduce the network's size by over 60%, which is huge. However, this results in a test accuracy drop of only 0.07%, which is insignificant.

As we increase the threshold, the number of neurons dropped also increases. But that also results in a consistent drop in test accuracy.

What's surprising and somewhat counterintuitive, however, is that the inference time remains almost the same.

Here, we must note that reducing the number of parameters in a neural network through activation pruning can indeed lead to a smaller model in terms of parameter count.

**However, the reduction in inference run-time may not always be directly proportional to the decrease in parameters.**

Matrix multiplications are quite optimized in frameworks like PyTorch. Thus, as long as matrices are small (which is true in our case), they rarely introduce significant overheads.

In our case, the run-time remains almost the same because the pruned network is still five layers deep, and PyTorch still has to iteratively go over each and every layer to compute the final prediction.

Nonetheless, activation pruning is still pretty reliable for reducing the size of the network by eliminating redundant neurons.

|                | Threshold | Accuracy | Inference Time | Size Reduction |
|----------------|-----------|----------|----------------|----------------|
| Original Model | 0         | 96.69    | 1.18           | 0%             |
| Pruned Model   | 0.1       | 96.62    | 1.09           | 62%            |
|                | 0.2       | 96.41    | 1.01           | 66%            |
|                | 0.3       | 96.31    | 1.28           | 69%            |
|                | 0.4       | 96.08    | 1.19           | 72%            |
|                | 0.5       | 94.88    | 1.10           | 74%            |
|                | 0.6       | 94.49    | 1.08           | 76%            |
|                | 0.7       | 92.19    | 1.22           | 79%            |
|                | 0.8       | 90.65    | 1.30           | 81%            |
|                | 0.9       | 88.39    | 1.21           | 83%            |
|                | 1         | 87.87    | 1.30           | 84%            |

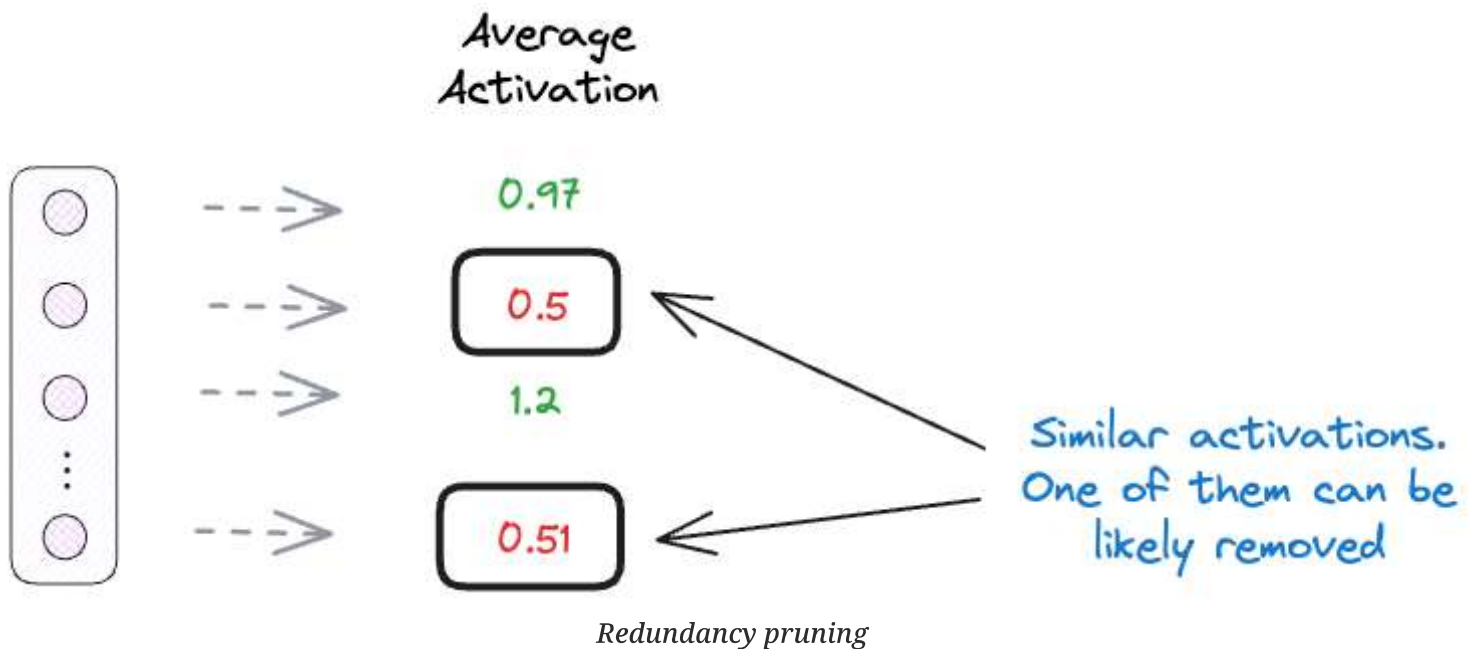
*Results of activation pruning*

As we noticed above, there isn't a significant drop in accuracy until  $\lambda = 0.4$ , but the size of the network still reduces by 70%.

## #2.3) Redundancy pruning

At times, if two neurons in a layer have very similar activations, it can mean they are capturing the same features.

By this intuition, we may remove one of the neurons and preserve the same functionality.



While this sounds quite intuitive, this is a bit complex to implement.

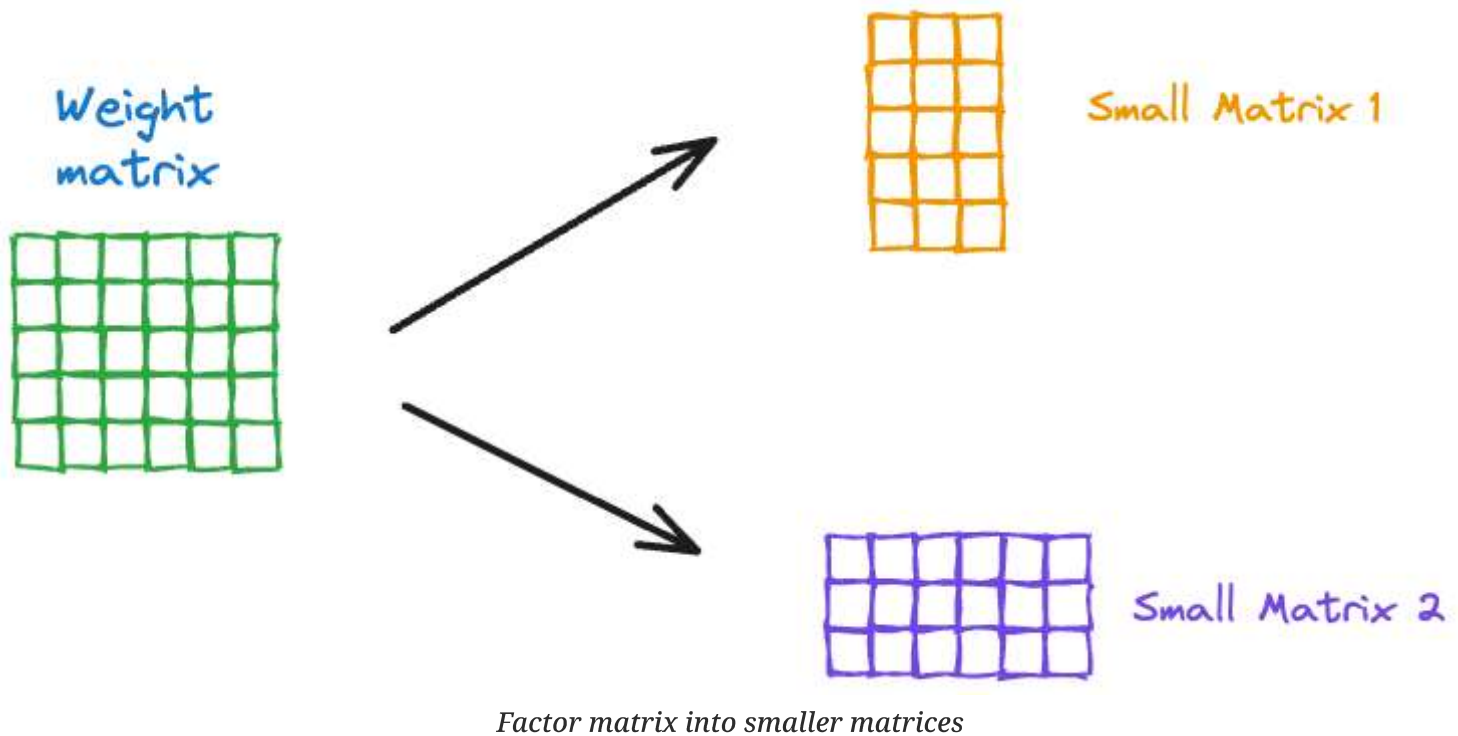
Also, redundancy pruning, at times, can be **a bit unreliable** way to prune neural networks. Can you answer why? This is an exercise for you. Let me know your response :)

Nonetheless, I think it is still useful to be aware of common techniques proposed in the literature.

### #3) Low-rank Factorization

At its core, Low-rank Factorization aims to approximate the weight matrices of neural networks using lower-rank matrices.

Essentially, the idea is to represent complex weight matrices as products of two or more simpler matrices.



The idea will become more clear if we understand these individual terms:

- **Low-rank:**

- In linear algebra, the “rank” of a matrix refers to the maximum number of linearly independent rows (or columns) in that matrix.
- Thus, a matrix with a low rank has fewer linearly independent rows or columns.
- This implies that it can be approximated as a combination of a smaller number of basis vectors, helping us to reduce the dimensionality.

- **Factorization**

- In mathematics, “factorization” means expressing a complex mathematical object (such as a matrix) as the product of simpler mathematical objects.

Thus, **Low-rank Factorization** means breaking down a given weight matrix into the product of two or more matrices of lower dimensions.

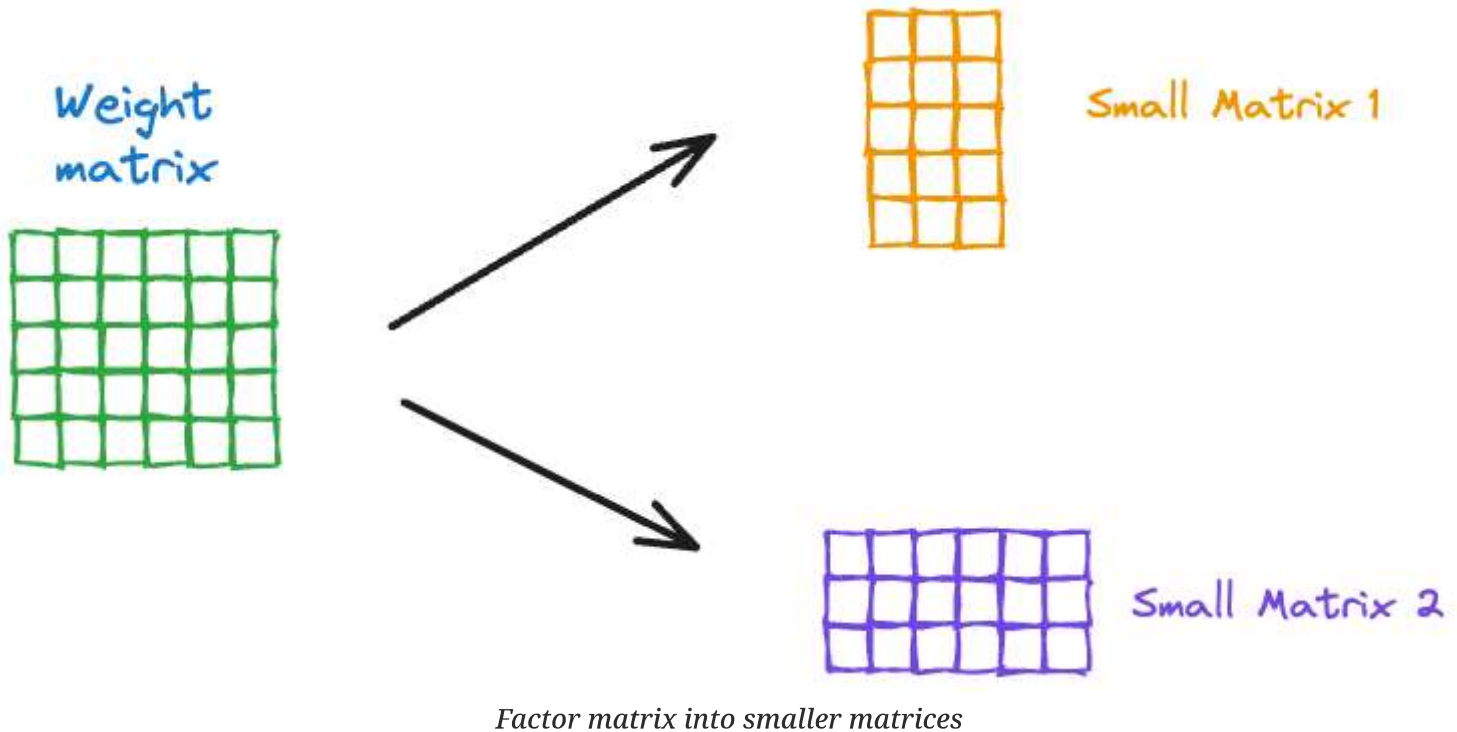
These lower-dimensional matrices are often called "factor matrices" or "basis matrices."

Let's understand how it works.

## Step 1) Perform Matrix Factorization

In a neural network, every layer will have a weight matrix.

We can decompose these original weight matrices into lower-rank approximations.

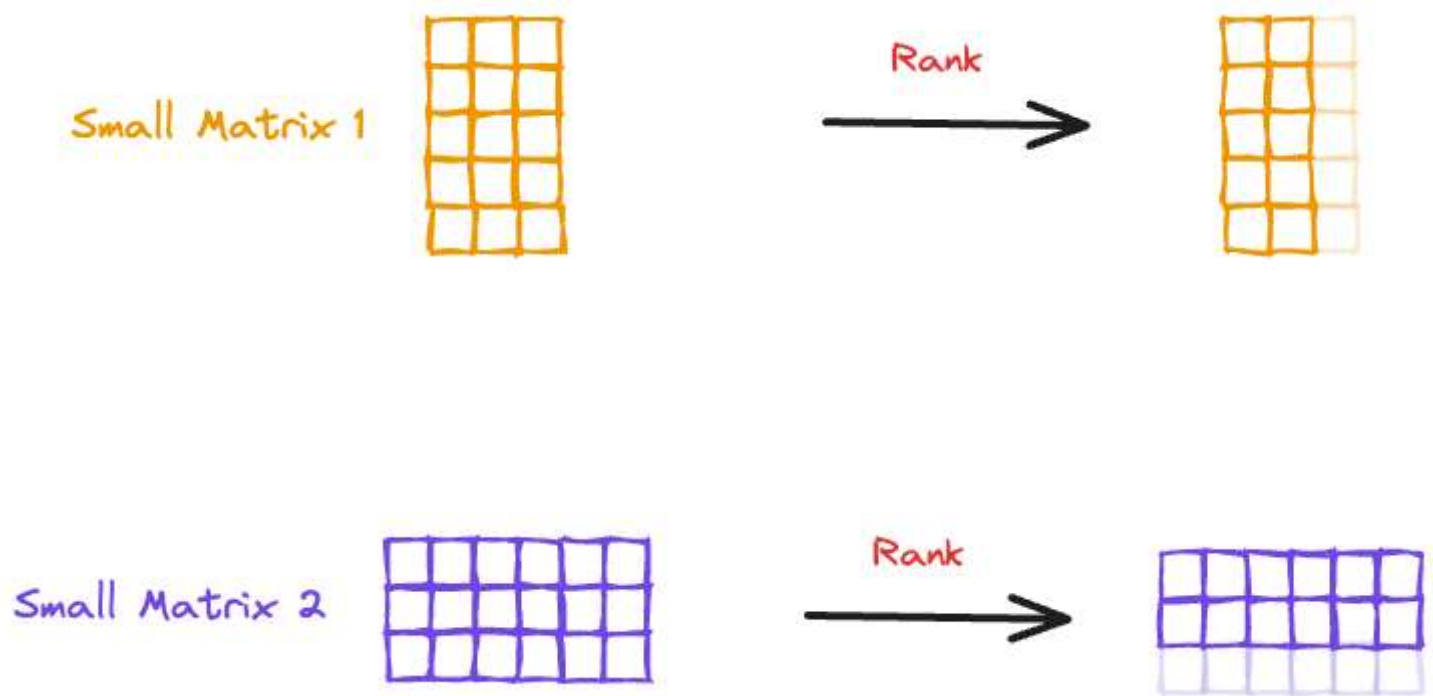


There are many different matrix factorization methods available, such as Singular Value Decomposition (SVD), Non-negative Matrix Factorization (NMF), or Truncated SVD.

## Step 2) Specify Rank

In matrix factorization, you'll typically have to choose the rank  $k$  for the lower-rank approximation.

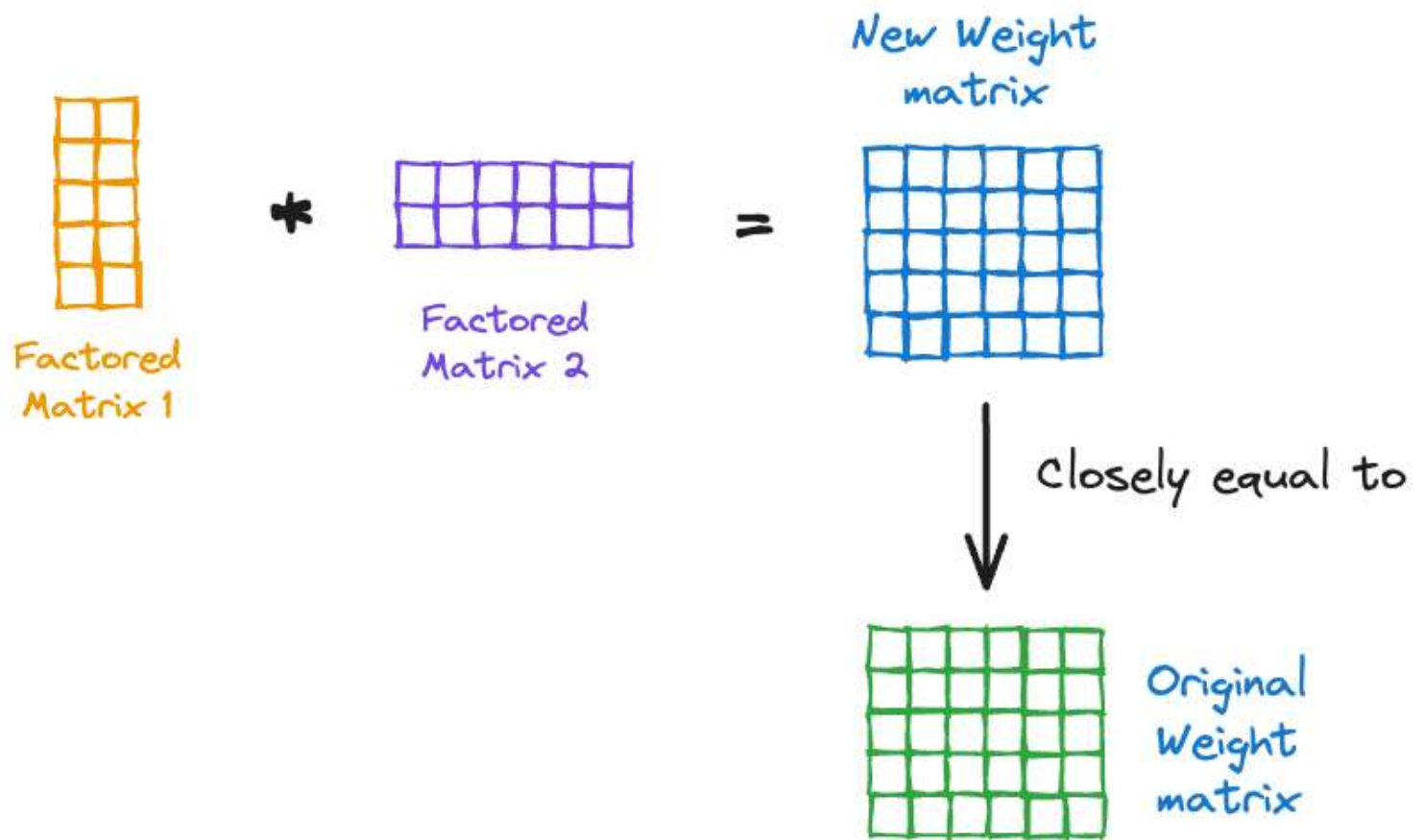
It determines the number of singular values (for SVD) or the number of factors (for factorization methods).



The choice of rank  $k$  is directly linked to the trade-off between model size reduction and preservation of information.

## Step 3) Reconstruct the Weight Matrices

Once you've obtained the lower-rank matrices, you can use them to transform the input instead of the original weight matrices.



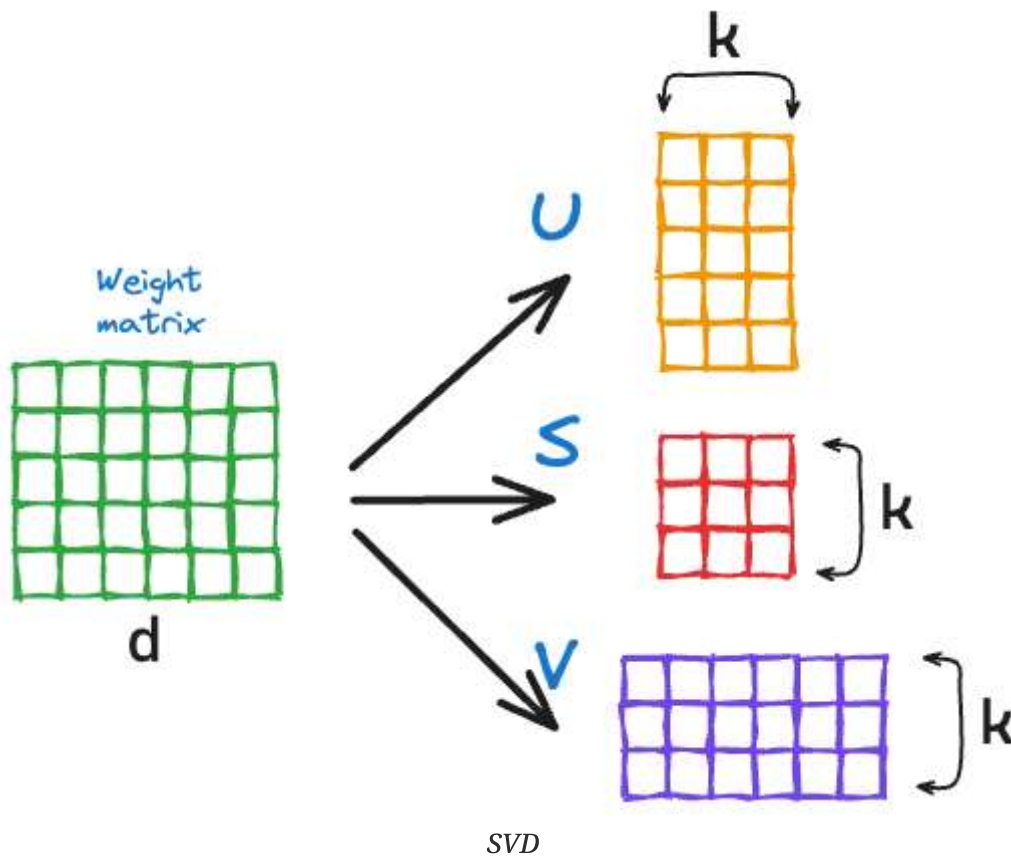
The benefit of doing this is that it reduces the computational complexity of the neural network while retaining important features learned during training.

By replacing the original weight matrices with their lower-rank approximations, we can effectively reduce the number of parameters in the model, which reduces its size.

### Let's see this in action!

We shall use [Singular Value Decomposition \(SVD\)](#) to factor the weight matrices into low-rank matrices.

While covering what SVD is beyond the scope of this article, the gist behind SVD is to decompose a matrix into three simpler matrices - U, S, and V, such that the original matrix can be reconstructed as the product of these three matrices.



In the above illustration,  $k$  denotes the rank, which is typically kept smaller than the original dimensionality ( $d$ ) of the data.

The choice of rank  $k$  directly linked to the trade-off between dimensionality reduction and preservation of information.

The higher the rank ( $k$ ), more the information preserved.

Let's see how we can use SVD for low-rank factorization of our weight matrices.

For simplicity, we shall consider that we have already trained our neural network:

```
class MyNeuralNetwork(nn.Module):  
    def __init__(self):  
        super(SimpleNet, self).__init__()  
        self.fc1 = nn.Linear(28 * 28, 512)  
        self.fc2 = nn.Linear(512, 256)  
        self.fc3 = nn.Linear(256, 128)  
        self.fc4 = nn.Linear(128, 10)  
  
    def forward(self, x):  
        ...  
        return x  
  
# Initialize the network  
net = MyNeuralNetwork()  
  
# Train the network (same as before)  
...
```

Train the  
Neural  
Network



Let's decompose the weights of only one layer, say, `self.fc3` to practically demonstrate low-rank factorization.

👉 You can replicate the upcoming steps for all other layers too. We are only decomposing one layer for simplicity.

We invoke the `torch.svd()` method on the weights of layer `fc3`.



Printing the shape of the weight matrix and the decomposed matrices, we see that the dimensions are the same — 128.

This is where we introduce the rank parameter (`k`) to truncate `U`, `S`, and `V` and retain only the top `k` components.

This is demonstrated below:



```
# Define rank
rank = 64

# Truncate U, S, and V
U_low_rank = U[:, :rank]
S_low_rank = torch.diag(S[:rank])
V_low_rank = V[:, :rank]
```

Done!

Now, if we print their shapes, we get:

```
print(U_low_rank.shape) # torch.Size([128, 64])
print(S_low_rank.shape) # torch.Size([64, 64])
print(V_low_rank.shape) # torch.Size([256, 64])
```

 Copy

As depicted above, the dimension of the decomposed matrices is smaller than what we had earlier.

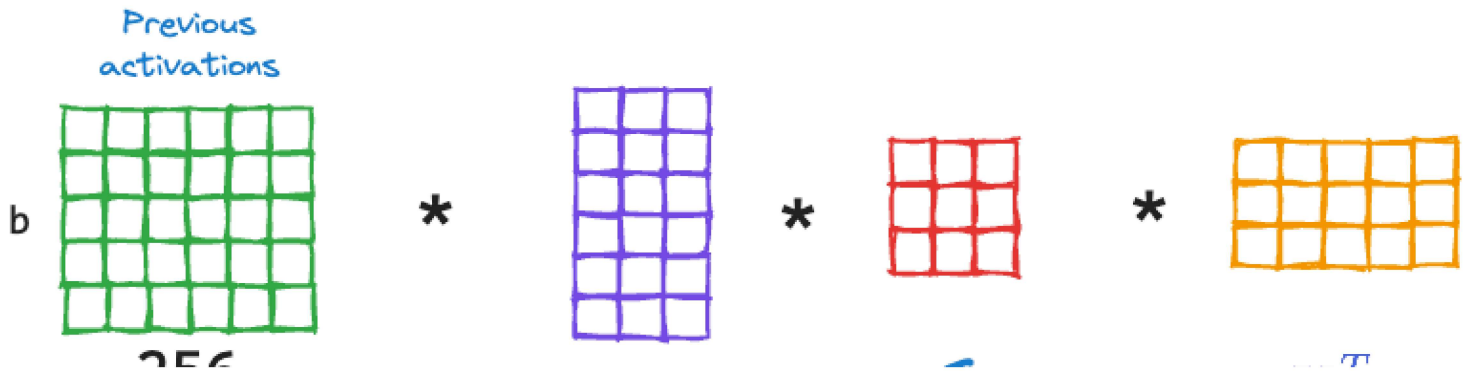
Now that we have decomposed the weights matrices of `net.fc3` into smaller matrices, we can use the smaller matrices to compute the activations as follows:

In our network, the layer `net.fc3` receives input from the layer `net.fc2`.



Thus, the input dimensions of the layer `net.fc3` will be  $(b, 256)$ , where `b` denotes the batch size.

Finally, instead of using the weights from `net.fc3`, we can use the matrices `U`, `S`, and `V` to compute the activations of that layer as follows:



Here, you might be wondering...how does that help?

Firstly, let's compute the total number of parameters pre and post-low-rank decomposition, specifically for the layer `net.fc3`.

Decomposing the weight matrix leads to a decrease in the total number of parameters.

Of course, the difference is small. Nonetheless, we did save some memory.

Next, we shall calculate the total number of operations performed:

- Using the usual weight matrix.

- Using decomposed matrices.

Let's assume that the input of the `net.fc3` layer is  $(b, 256)$ , where `b` denotes the batch size (set to 32 for our case).

If we were to use the original weight matrix of `net.fc3`, it would have resulted in:

*Number of operations using the original weight matrix*



Multiplying two matrices of shape  $(A, B)$  and  $(B, C)$  results in  $A*B*C$  operations.

Now, let's compute the same if we would have used low-rank factorization instead.

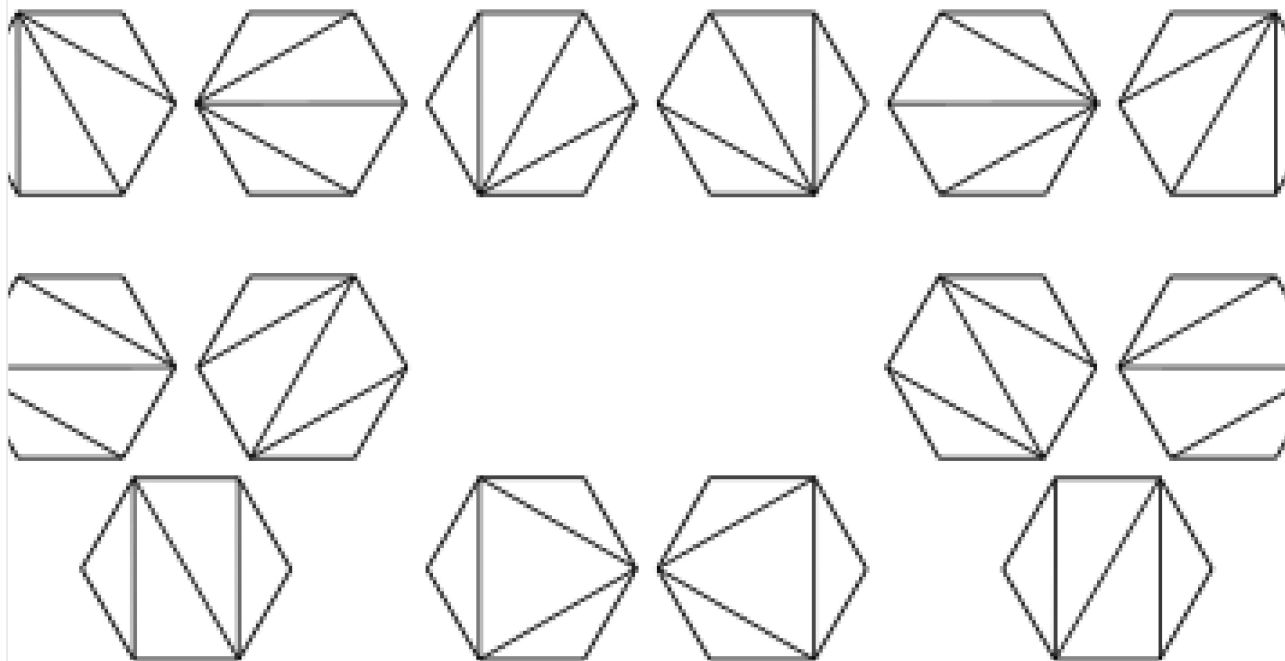
There are various ways to compute the above matrix multiplication.

For instance, if we intend to multiply four matrices ( $A * B * C * D$ ), there are many possible to do so:

- $((A * B) * (C * D))$
- $(A * (B * C) * D)$
- $A * (B * (C * D))$
- and many more.

The number of operations will differ depending on the choice of order of multiplication.

The objective is to obtain the result by performing minimum operations. One optimal way to solve this is by using dynamic programming. We won't get into much detail here, but you can read more about it below:



## Matrix chain multiplication - Wikipedia

W Wikimedia Foundation, Inc. • Contributors to Wikimedia pr...

In our case, the most optimal way is as follows:

*Number of operations using the factorized matrices*

This results in an approximate decrease of 15% in the total number of operations, which, again, is impressive.

But we need to verify if this was even helpful or not.

In other words, we must evaluate the model's accuracy and compare it with its accuracy before factorization.

Similar to what we did in activation pruning, let's evaluate it across a range of rank (`k`) values.

We shall measure:

- The accuracy of the model after weight factorization of `net.fc3` layer only.
- The reduction in number of operations for that layer.

The results are shown below:

*Results of Low-rank factorization*

The results look pretty impressive, don't they?

Here are a few things to notice:

- For large values of rank (100, 90, 80):
  - Factorization results in a slight increase in accuracy. It could be because factorization helped us eliminate features that could have negatively impacted the model.
  - At the same time, it does result in an increase in total operations, which is undesired. This becomes more concerning because it only results in a minute increase in accuracy.
- At low values of rank (5, 2, 1):
  - The accuracy drops drastically.
  - This is expected as we may have lost plenty of original information — *"The choice of rank  $k$  is linked to the trade-off between model size reduction and preservation of information."*
- According to me, the most impressive results appear at a rank value of 10:
  - The model's accuracy is almost the same as that of the original model.
  - But it results in a decrease in total number of operations by 88%, which is huge.

**Here, we must consider that we only decomposed one layer for now. We can factorize other weight matrices as well to decrease the model size and number of operations even further.**

Overall, the task always boils down to subjectively and empirically evaluating the model across a range of rank values and selecting the results that appear to be most impressive.

## #4) Quantization

Lastly, we shall discuss yet another intuitive and useful technique to reduce the model's size.

# Motivation

Typically, the parameters of a neural network (layer weights) are represented using 32-bit floating-point numbers. This is useful because it offers a high level of precision.

Also, as parameters are typically not constrained to any specific range of values, all deep learning frameworks, by default, assign the biggest data type to parameters.

But using the biggest data type also means consuming more memory.

As you may have guessed, Quantization involves using lower-bit representations, such as 16-bit, 8-bit, 4-bit, or even 1-bit, to represent parameters.

This results in a significant decrease in the amount of memory required to store the model's parameters.

For instance, consider your model has over a million parameters, each represented with 32-bit floating-point numbers.

If possible, representing them 8-bit numbers can result in a significant decrease (~75%) in memory usage while still allowing for a large range of values to be

represented.

Of course, Quantization introduces a trade-off between model size and precision.

While reducing the bit-width of parameters makes the model smaller, it also leads to a loss of precision.

This means the model's predictions become more approximate than the original, full-precision model.

Thus, it is important to carefully evaluate the trade-off between model size/inference speed and accuracy when considering Quantization for deployment.

Despite this trade-off, Quantization can be particularly useful in scenarios where model size is a critical constraint, such as edge devices, mobile applications, or specialized hardware like smartphones.

One of the most common ways Quantization is practically used is called **Post-Training Quantization**.

## Post-Training Quantization

As the name suggests, this involves representing the weights of a trained model using lower-precision floating numbers.

So first, we train the model as we typically would.

Next, we change the data type of model parameters.

This is done by evaluating the model's weights and activations on a representative dataset and determining the most suitable precision to represent them.

Let's assume that we have already trained a model in PyTorch `net`. To reiterate, it is the same model that we have discussing throughout this entire article:

```
class MyNeuralNetwork(nn.Module):  
    def __init__(self):  
        super(SimpleNet, self).__init__()  
        self.fc1 = nn.Linear(28 * 28, 512)  
        self.fc2 = nn.Linear(512, 256)  
        self.fc3 = nn.Linear(256, 128)  
        self.fc4 = nn.Linear(128, 10)  
  
    def forward(self, x):  
        ...  
        return x  
  
# Initialize the network  
net = MyNeuralNetwork()  
  
# Train the network (same as before)  
...
```

**Train the  
Neural  
Network**



In PyTorch, we can leverage post-training Quantization using its `torch.quantization` package.

This is demonstrated below:



## Post-training Quantization

```
# Convert the model to quantized version
quantized_model = torch.quantization.quantize_dynamic(
    net, # Your model
    {torch.nn.Linear}, # The layers to be quantized
    dtype=torch.qint8 # Quantization data type (int8)
)
```

In this code:

- We Use `torch.quantization.quantize_dynamic` method to quantize the trained model.
- In this case, we quantize the linear ( `torch.nn.Linear` ) layers of the model to `int8` ( `torch.qint8` ) data type. You can specify the layers to be quantized based on your requirements.

Done!

Now, the model has been quantized, and we can use it for inference.

As depicted above:

- Both models achieve the same test accuracy.
- However, the quantized model is smaller because it uses a smaller data type to represent the parameters.



While in this case, the accuracy is the same. However, it does not mean that a quantized model will always perform as well as the unquantized (or normal) model. It is important to evaluate the quantized model across a range of data types and subjectively decide the best model as we did before in zero pruning, activation pruning, and low-rank factorization.

## Conclusion

And with this, we come to the end of this article.

In this article, we discussed several model compression techniques that can be useful to:

1. Speed up the model inference phase
2. Reduce the model's size

Both parameters mentioned above are a critical requirement for models deployed in production.

In particular, we discussed:

- **Knowledge distillation:** Train a large model (teacher) and transfer its knowledge to a smaller model (student).
- **Pruning:** Remove irrelevant edges and nodes from a network.
  - Zero pruning
  - Activation pruning
  - Redundancy pruning
- **Low-rank Factorization:** Decompose weight matrices into smaller “low-rank” matrices.
- **Quantization:** Using lower-bit representation to store parameters to reduce the model's memory usage instead of the default large representations.

We also discussed their implementations in Python.

You can find the code for this article here:

**Model Compression Article**



Model Compression Article.ipynb • 48 KB

Being aware of such techniques is also immensely helpful in deploying ML models on devices with few resources — smartphones and Amazon Alexa, for instance.

I hope this article was helpful and you learned something new.

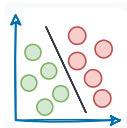
Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:

*Connect via chat*



# Daily Dose of Data Science

A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

Machine Learning

Share this article



Read next

LLMs Feb 18, 2024 • 47 min read



## A Beginner-friendly and Comprehensive Deep Dive on Vector Databases

Understanding every little detail on vector databases and their utility in LLMs, along with a hands-on demo.

Avi Chawla

---

Deployment Oct 23, 2023 • 32 min read

## You Cannot Build Large Data Projects Until You Learn Data Version Control!

The underappreciated, yet critical, skill that most data scientists overlook.

Avi Chawla

---

Machine Learning

Mar 12, 2024 • 22 min read



## A Detailed and Beginner-Friendly Introduction to PyTorch Lightning: The Supercharged PyTorch

Immensely simplify deep learning model building with PyTorch Lightning.

Avi Chawla

### Join the Daily Dose of Data Science Today!

A daily column with insights, observations, tutorials, and best practices on data science.

[Get Started!](#)



A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students, across:



## Navigation

[Newsletter](#)[Blogs](#)[More](#)[Contact](#)[FAQs](#)[About](#)[Search](#) [Sign in](#)[Get Started](#)